MAJOR PROJECT REPORT

on

ADVANCED GATE PASS SYSTEM (GateVault)

A Web-Based, Mobile-Responsive Role-Based Gate Pass & Leave Management System

Submitted in partial fulfillment of requirements for the award of

Bachelor of Technology (B.Tech.)

in

Computer Science and Engineering

KAZIRANGA UNIVERSITY
KNOWLEDGE & BEYOND

Submitted by:

Name of the students	Student ID
Ananda Saikia	**ET22BTHCS072**
Ashish Iqbal	**ET22BTHCS118**
Bhakti Prasad Mohan	**ET22BTHCS059**
Pranjit Malakar	**ET22BTHCS073**
Sarmistha Saikia	**ET22BTHCS024**

Under the Supervision and Guidance of

Dr. Samik Chakraborty
Professor, Department of CSE

Mr. Faizul Haque
Assistant Professor, Department of SCS IT

Department of Computer Science and Engineering
School of Engineering and Technology
The Assam Kaziranga University, Jorhat, Assam

May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEEING
SCHOOL OF ENGINEERING AND TECHNOLOGY
THE ASSAM KAZIRANGA UNIVERSITY
JORHAT-785006 :: ASSAM :: INDIA

CERTIFICATE

This is to certify that the project report entitled ***"ADVANCED GATE PASS SYSTEM (GateVault) A Web-Based, Mobile-Responsive Role-Based Gate Pass & Leave Management System"***, submitted to the Department of Computer Science and Engineering, The Assam Kaziranga University, in partial fulfillment for the award of the degree of ***Bachelor of Technology in Computer Science and Engineering***, is a record of bona fide work carried out by ***Mr Ananda Saikia (ET22BTHCS072), Mr Ashish Iqbal (ET22BTHCS118), Mr Bhakti Prasad Mohan (ET22BTHCS059), Mr Pranjit Malakar (ET22BTHCS073), Miss Sarmistha Saikia (ET22BTHCS024)*** under our supervision and guidance.

All help received by us from various sources have been duly acknowledged.
No part of this report has been submitted elsewhere for award of any other degree.

.....	.....
Dr. Samik Chakraborty Professor Department of CSE	Mr. Faizul Haque Assistant Professor Department of SCS IT
.....	.....
Dr. Mousoomi Bora Associate Professor Head of the Department, CSE	Dr. Ripunjoy Gogoi Associate Professor Dean of SET

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our respected project guides, **Prof. Samik Chakraborty** and **Mr. Faizul Haque**, for their constant guidance, valuable suggestions, and continuous encouragement throughout the development of our project titled **"Advanced Gate Pass System."** Their technical expertise and support played a significant role in the successful completion of this work.

We would also like to sincerely thank **Ms. Tandrali Ray**, Assistant Professor**,** Department of Computer Science & Engineering, for her valuable support, encouragement, and guidance during the preparation and completion of this project report. Her suggestions and motivation greatly helped us improve our work.

We are thankful to the **Department of Computer Science & Engineering, School of Engineering and Technology, The Assam Kaziranga University**, for providing us with the opportunity, facilities, and a supportive environment to successfully carry out this project.

We would also like to extend our heartfelt thanks to all the faculty members, classmates, and friends who directly or indirectly supported us with their ideas, suggestions, and encouragement throughout the project journey.

Finally, we express our deepest gratitude to our parents and family members for their continuous motivation, patience, and moral support throughout our academic journey and during the successful completion of this project.

DECLARATION

We hereby declare that the project work entitled **"Advanced Gate Pass System"** is an original work carried out by us under the supervision of **Prof. Samik Chakraborty** and **Mr. Faizul Haque**, Department of Computer Science & Engineering, School of Engineering and Technology, The Assam Kaziranga University.

This project report is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.) in Computer Science & Engineering**. We further declare that the work presented in this report has been carried out by our team and has not been submitted previously to any other university or institution for the award of any degree, diploma, or other academic qualification.

All sources of information used in this project have been properly acknowledged and referenced wherever necessary.

Name of the students	Student ID	Signature of Student
1. Ananda Saikia	ET22BTHCS072	
2. Ashish Iqbal	ET22BTHCS118	
3. Bhakti Prasad Mohan	ET22BTHCS059	
4. Pranjit Malakar	ET22BTHCS073	
5. Sarmistha Saikia	ET22BTHCS024	

Date: 25/05/2026
Place: Jorhat, Assam

ABSTRACT

The **Advanced Gate Pass System (GateVault)** is a modern web-based and mobile responsive application designed to digitize and automate the traditional gate pass and leave approval process used in educational institutions. The existing manual system, which depends on handwritten forms and physical signatures, is often time-consuming, inefficient, and vulnerable to errors or misuse. This project aims to provide a secure, fast, and transparent solution by replacing the conventional paper-based process with a centralized digital platform.

The system provides separate dashboards and role-based access for Students, Hostel Wardens, Heads of Departments (HoDs), Security Guards, and Administrators. Students can submit gate pass or leave requests online, track approval status in real-time, and generate temporary QR codes for verification. Hostel Wardens and HoDs can efficiently review, approve, or reject requests through their respective dashboards, while Security Guards can verify requests instantly using QR code scanning.

The project is developed using modern full-stack technologies such as **Next.js, React.js, TypeScript, Tailwind CSS, MongoDB, NextAuth.js, and QR-based verification mechanisms**. The implementation of QR code authentication enhances security by preventing unauthorized access biased and fake gate passes.

The proposed system improves operational efficiency, transparency, security, and centralized record management within institutions. It also demonstrates the practical use of modern web technologies in solving real-world administrative challenges and has strong potential for future enhancements such as AI-based decision making, biometric integration, and automated notifications.

Keywords: Advanced Gate Pass System, QR Code Verification, Leave Management System, Role-Based Access Control, Next.js, MongoDB, Web Application, Campus Security, Digital Approval System, Full Stack Development.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

Chapter 1

Introduction

1.1 Overview of the Project

Managing student movement within a college campus — particularly for hostel residents — has always been an administrative responsibility that institutions take seriously. When a student needs to leave the campus for personal reasons or during holidays, the institution must have a clear record of who has left, when, under whose approval, and when they returned. This responsibility, though straightforward in principle, becomes difficult to manage efficiently when handled through paper-based processes.

GateVault, formally titled the Advanced Gate Pass System, is a web-based and mobile-responsive application developed to address this challenge for educational institutions. The system digitises the entire gate pass and leave approval process, replacing handwritten forms and physical signatures with a structured, role-based digital workflow. It is designed to work across five distinct user roles — Student, Head of Department (HoD), Hostel Warden, Security Guard, and Administrator — each with a dedicated interface suited to their specific responsibilities within the process.

A student who wishes to leave the campus can log into the system, fill out either a Gate Pass form or a Leave form depending on the nature of the request, and submit it digitally. The request is then automatically routed to the appropriate authority — the Warden for gate passes, and the HoD followed by the Warden for leave requests on working days. Once the request is approved, the system generates a temporary QR code linked to that specific request. The student presents this QR code at the gate, where the Security Guard scans it to verify the approval status in real time before permitting or denying exit.

The application is built using Next.js and React for the frontend, with TypeScript ensuring type safety throughout the codebase. Tailwind CSS handles the responsive styling, allowing the interface to function equally well on mobile devices and desktop browsers. The backend is powered by Next.js API routes, with authentication managed through NextAuth.js and password security handled via bcryptjs. All data is stored in MongoDB through Mongoose. The application is deployed on Vercel and is accessible at *https://gatevault-agps.vercel.app*.

The project was conceived and developed by a team of five final year undergraduate students from the Department of Computer Science and Engineering, The Assam Kaziranga University, as part of the curriculum requirements for the degree of Bachelor of Technology.

1.2 Problem Statement

Most engineering and general degree colleges in India still follow a manual process for gate pass and leave approval. When a hostel student needs to go home or step out of campus, the typical procedure involves collecting a printed or handwritten form, getting it signed by the warden, and in some cases also obtaining the signature of the Head of Department. The student must physically visit each authority, often during their working hours, and wait for their availability before the form can be processed.

This process creates several practical problems that affect students and staff alike.

The most immediate issue is the time involved. During peak periods such as festival breaks or semester-end holidays, a large number of students attempt to obtain gate passes within a short window of time. Faculty members and wardens are then faced with a pile of physical forms, and students are left waiting for hours without any clarity on when their request will be addressed.

Faculty unavailability adds another layer of difficulty. If an HoD is in a meeting, conducting a class, or absent on a particular day, the entire approval chain stalls. As well as during night time the respective authority may not be available. There is no mechanism for the student to know when the authority will be available, and no alternative path for the request to proceed in the meantime.

Paper-based forms also carry an inherent risk of misuse. Signatures can be forged, forms can be altered after signing, and printed slips can be easily duplicated. Security guards stationed at the gate have no reliable way to verify whether a form presented to them is genuine, since they are expected to make a judgment call based purely on the appearance of the document and the signature on it.

Beyond the security concern, there is also the problem of record-keeping. Paper forms are prone to being lost, damaged, or misfiled. When an institution needs to review the movement history of a student — particularly in cases involving disciplinary matters or safety concerns — retrieving accurate records from a collection of physical forms is time-consuming and often incomplete.

Finally, there is no transparency for the student in this process. Once a form is submitted, the student has no way to know where their request currently stands — whether it has been seen, approved, or is still waiting. This uncertainty is a source of unnecessary stress, particularly when a student has travel bookings or time-sensitive plans dependent on the approval.

These problems together point to the need for a centralised, digital, and role-aware system that can handle the full lifecycle of a gate pass or leave request in a structured and verifiable manner.

1.3 Objectives of the Project

The following objectives were defined at the outset of the project and guided the design and development of the system throughout.

The primary objective was to build a paperless, digital gate pass and leave management system that can handle the full approval workflow from the moment a student submits a request to the point of physical verification at the gate.

The system was further designed to provide a role-specific experience for each type of user. Students, HoDs, Wardens, Security Guards, and Administrators each interact with the system through an interface tailored to their responsibilities, ensuring that no user has access to functions outside their role.

Real-time status visibility was a core objective. Students should be able to see the current status of their request — whether it is pending, approved, or denied — without having to contact any authority directly.

The system aimed to introduce QR code-based verification at the gate as a fraud-resistant alternative to paper slips. Each approved request generates a temporary QR code that is unique to that request and is the only valid basis on which the Security Guard permits a student to exit.

The project also aimed to maintain a complete and searchable history of all requests, approvals, and gate exits, so that administrators and wardens can retrieve records whenever required.

Finally, the system was intended to be mobile-responsive, ensuring that students and staff can use it on their smartphones without any loss of functionality.

1.4 Scope of the Project

The scope of this project is limited to gate pass and leave request management within a single educational institution operating with a hostel facility. The system manages two types of requests — Gate Pass, for short-term exit within the same day, and Leave, for overnight or

multi-day absence — and routes them through the appropriate approval hierarchy based on the type and timing of the request.

The system covers five user roles: Student, HoD, Hostel Warden, Security Guard, and Administrator. Each role is handled through a separate login and dashboard. The Administrator role provides oversight of the entire system, including the ability to view all requests and user activity across the institution.

The scope includes the generation and scanning of QR codes for gate verification, maintenance of request history logs for all roles, and a basic search functionality that allows wardens and administrators to look up requests by student name.

The current version of the system does not include integration with any external ERP or student management system, SMS or WhatsApp notification services, biometric verification, or multi-campus support. These remain identified areas for future development and are discussed in Chapter 8.

1.5 Existing System

The existing gate pass management process in most educational institutions, including the context for which this system was designed, is entirely manual. A student who wishes to leave the campus approaches the hostel warden with a handwritten or printed request form. The form typically includes the student's name, roll number, reason for leaving, destination, and expected date and time of return.

If the request is for a leave period that falls on a working day, the student is additionally required to obtain the signature of the Head of Department before approaching the warden. This means the student must first visit the department office, wait for the HoD's availability, and then return to the hostel office for the warden's approval.

Once both signatures are obtained, the form is retained by the warden and a copy or a handwritten slip is given to the student to present at the gate. The Security Guard at the gate inspects the slip, checks the signatures visually, and allows the student to pass if the form appears to be in order.

On return, the student is required to report to the warden's office to complete the entry record. This step is frequently skipped or loosely enforced, which means the institution often has incomplete records of student return timings.

The limitations of this process are well recognised. There is no digital trail for any part of the workflow. Approval depends entirely on the physical presence of the authorities concerned. The student has no way to track the status of a pending request. Security guards have no means of verifying the authenticity of the document presented to them. And the institution has no centralised, searchable log of movement records that can be accessed when needed.

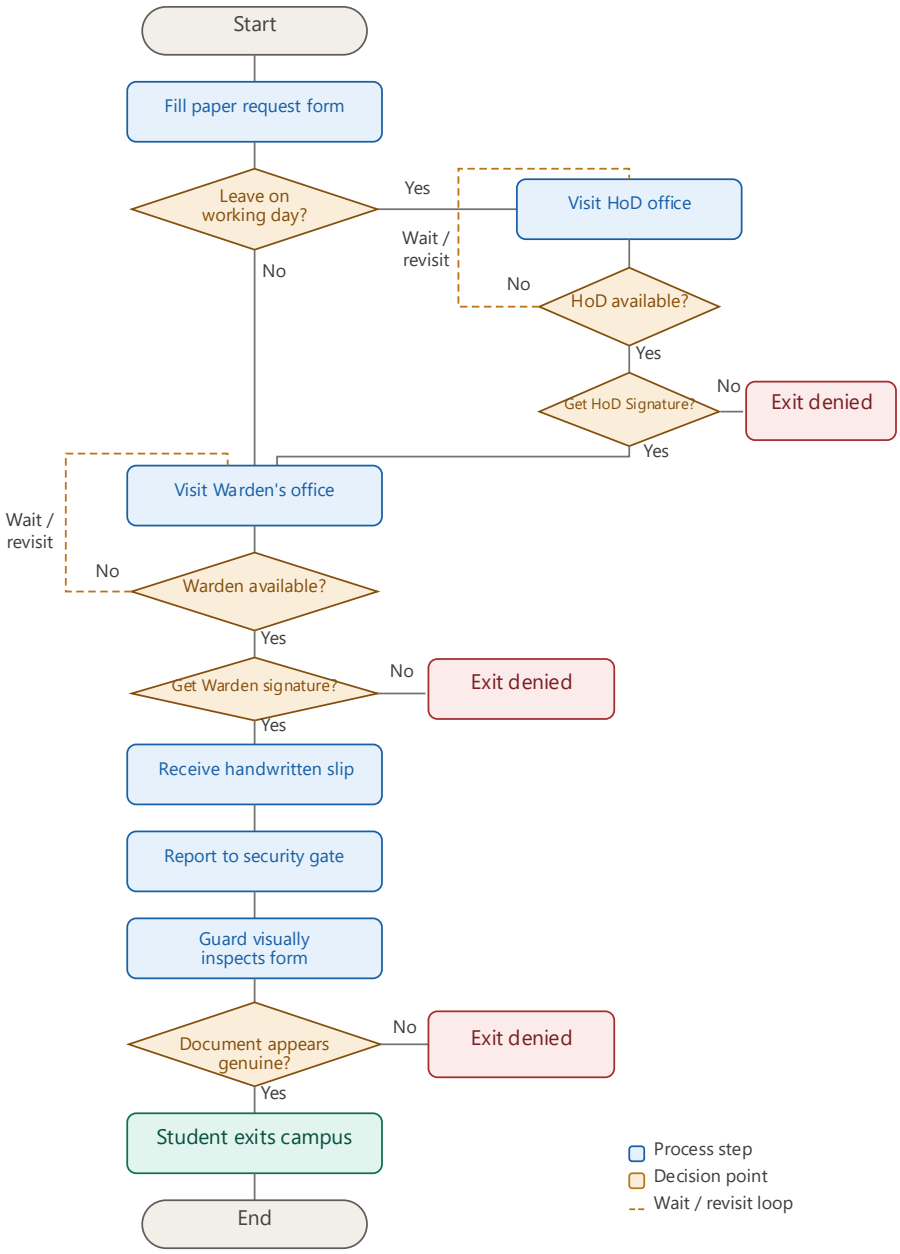

Figure 1.1: Flowchart of the Existing Manual Gate Pass Process

1.6 Proposed System

The Advanced Gate Pass System — GateVault — proposes a fully digital replacement for the manual process described above. The system is designed so that every step of the gate pass or leave request lifecycle, from submission to verification, takes place through a web-based interface that any authorised user can access from a browser on their phone or computer.

When a student needs to leave the campus, they log into the system using their registered credentials and navigate to the form submission section of their dashboard. They select the type of request — Gate Pass or Leave — fill in the required details, and submit the form. The system immediately assigns a unique Request ID to the submission and routes it to the appropriate authority based on the request type and date.

Gate Pass requests are sent directly to the Hostel Warden. Leave requests on non-holiday days are routed first to the HoD, and upon HoD approval, forwarded automatically to the Warden for the second stage of approval. The relevant authority receives the request in their dashboard and can approve or deny it with a single action. At every stage, the student's dashboard is updated in real time to reflect the current status of the request.

Once a request receives full approval, the system generates a temporary QR code linked to that specific request. The student opens the QR code from their dashboard and presents it to the Security Guard at the gate. The guard scans the QR code using the scanning interface on their own dashboard, which immediately displays the status of the request — Approved, Denied, or Pending. Exit is permitted only for approved requests.

All request data, approval actions, and gate scan events are stored in the database and remain accessible through the history and log sections of each user's dashboard. Administrators can view all records across all students at any time.

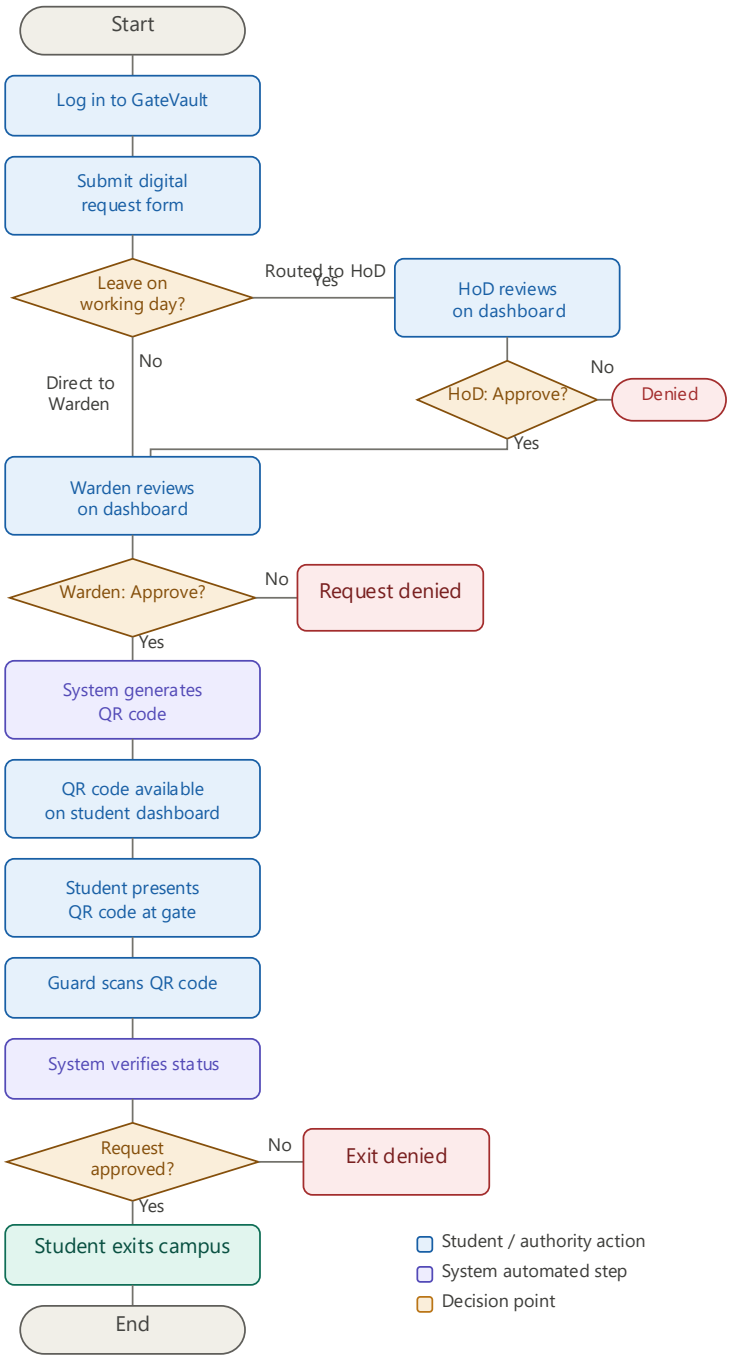

Figure 1.2: Flowchart of the Proposed Digital Gate Pass Process

1.7 Advantages of the Proposed System

The proposed system addresses each of the shortcomings identified in the existing manual process and introduces several operational improvements that benefit both students and institution staff.

Approval decisions that previously required a student to physically locate and wait for an authority can now be completed by the HoD or Warden from any device at any time. This significantly reduces the time taken to process a request, particularly during high-volume periods such as festival breaks.

The QR code-based verification system eliminates the possibility of forged or duplicated gate passes. Each QR code is unique to a specific request and is generated only upon approval, meaning that a student cannot present a pass that was not legitimately approved through the system.

Every action taken within the system — form submission, approval or denial, QR scan — is recorded with a timestamp and stored against the student's account. This creates a complete and accurate movement history that can be retrieved by administrators and wardens whenever required, without dependence on physical files.

Students no longer need to follow up with authorities in person to check on their request. The status is visible on their dashboard at all times, which reduces unnecessary communication and makes the process more transparent.

The system enforces strict role-based access, ensuring that each user can only view and act on information relevant to their role. A student cannot see another student's requests, and a security guard has no access to the approval workflow. This separation protects student data and maintains the integrity of the administrative process.

Finally, because the system is web-based and mobile-responsive, it requires no dedicated hardware or software installation. Any user with a smartphone and an internet connection can access their dashboard, making adoption straightforward for institutions of varying technical capacity.

Chapter 2

Literature Review

The purpose of this chapter is to examine the existing body of work related to digital gate pass systems, QR code-based verification, and role-based access control in web applications. By reviewing what has already been studied and built in these areas, it becomes possible to identify the specific gaps that the current project aims to address.

2.1 Existing Digital Gate Pass Systems

The problem of managing student movement within educational campuses has received attention from researchers and developers across different institutions, particularly over the last decade. Several systems have been proposed and implemented at various levels, ranging from simple web forms to more structured multi-role applications.

Patel et al. [1] developed an online gate pass system for hostel management in which students could submit requests through a web portal and wardens could approve or reject them digitally. The system reduced the need for physical forms and maintained a database of all submissions. However, the study noted that the system relied on email notifications for status updates, which introduced delays and was dependent on connectivity and the user's email habits. There was no real-time dashboard for students to check the status of a pending request.

A similar system proposed by Sharma and Gupta [2] focused specifically on reducing the administrative workload on hostel wardens in large residential universities. Their system automated the routing of requests and maintained logs of student exits and returns. While it was effective in organising the data on the warden's side, it did not address the gate verification step. The approved form was still printed and presented physically at the gate, which left the process vulnerable to the same forgery risks as the manual system.

Kumar et al. [3] proposed a mobile application-based gate pass system that integrated with the institution's student database and allowed real-time approval tracking. The application was Android-only and required installation on both the student's and warden's devices, which limited its adoption across diverse device environments. The system also had no provision for a role hierarchy in which a second authority — such as an HoD — could be part of the approval chain before the warden.

What is evident from the existing work is that while several systems have attempted to digitise parts of the gate pass process, most address only one or two aspects of the problem — typically the submission and approval stages — without building a complete end-to-end solution that also covers gate-level verification and multi-role access.

2.2 QR-Based Verification Systems

The QR code, originally developed by Denso Wave in 1994 for automotive component tracking, has since found application in a wide range of identification and verification contexts [4]. Its two-dimensional structure allows it to encode significantly more data than a traditional barcode while remaining scannable by any standard camera.

In the context of identity and access verification, QR codes have been used in boarding passes, event ticketing, contactless payments, and vaccination certificates, among other applications. Researchers have studied their suitability for secure document verification, with particular attention to the challenges of preventing code duplication and ensuring that a given code remains valid only for its intended purpose and duration.

Jayashankar and Rao [5] examined the use of dynamic QR codes for student attendance verification in colleges. Their study found that dynamic codes — those generated at the time of a specific action rather than stored as static images — were significantly more resistant to misuse than static codes, since a dynamic code becomes invalid once its associated action has been completed or its time window has expired. This finding is directly relevant to gate pass verification, where a QR code must be tied to a specific, time-bound approved request.

Singh et al. [6] studied QR-based entry management in institutional premises and reported that security personnel found QR scanning faster and more reliable than manual document inspection. The study also noted that the psychological deterrent of a digital verification system reduced the frequency of forgery attempts, as students were aware that the code would be checked against a live database rather than inspected on paper.

One limitation commonly noted in the literature is that QR code verification systems require the verifying device — in this case, the security guard's phone or terminal — to have a stable internet connection at the point of scanning. Offline fallback mechanisms have been proposed in some systems, though these typically come with security trade-offs.

2.3 Role-Based Access Control in Web Applications

Role-Based Access Control, commonly referred to as RBAC, is a method of restricting system access to authorised users based on the roles they are assigned within an organisation. The conceptual framework for RBAC was formally described by Sandhu et al. [7] and later standardised by the National Institute of Standards and Technology (NIST) as a reference model for access control in information systems [8]. Under this model, permissions are not assigned directly to individual users but are attached to roles, and users acquire permissions by being assigned to one or more roles.

In the context of web applications, RBAC has become a standard architectural pattern for systems that serve multiple categories of users with different levels of access. Ferraiolo et al. [8] identified four core components in the NIST RBAC model: users, roles, permissions, and sessions. This framework maps naturally onto an educational institution's administrative structure, where a student, a warden, an HoD, and an administrator each have clearly defined responsibilities and should only be able to perform actions relevant to those responsibilities.

Several studies have examined the practical implementation of RBAC in web-based institutional systems. Ahmad and Hussain [9] implemented role-based access in a university examination management portal and found that clearly separated role interfaces reduced user errors and unauthorised access incidents during a six-month deployment period. The study emphasised that enforcing role boundaries at the backend API level — rather than only in the frontend UI — was essential to prevent circumvention through direct requests to the server.

In the domain of hostel and campus management systems, RBAC has been applied to varying degrees. Some systems assign only two roles — student and administrator — while others have attempted more granular structures. The challenge with more roles is managing the overlap in responsibilities, particularly in cases where the same request must pass through multiple approvers in sequence. Designing the role hierarchy so that permissions are enforced correctly at every stage of a multi-step workflow, without creating bottlenecks or gaps, remains a practical design challenge in applied systems.

2.4 Research Gap

The literature reviewed in the preceding sections reveals consistent progress in the individual components of a digital gate pass system — form submission, approval management, QR-based verification, and role-based access control. However, a clear gap emerges when one

looks for a system that integrates all of these components into a single, cohesive application designed specifically for the educational hostel management context.

Most existing digital gate pass systems stop at the approval stage and do not extend into gate-level verification. Those that do include some form of verification at the gate rely on printed outputs or static codes, both of which carry residual security risks. None of the reviewed systems implement a dynamic, request-specific QR code that is generated only upon approval and remains tied to a live database entry that is checked at the time of scanning.

Additionally, the majority of existing systems support only two or three user roles. The nuanced hierarchy present in Indian educational institutions — where a leave request on a working day must pass through the HoD before reaching the Warden, while a simple gate pass goes only to the Warden — is not addressed in the reviewed literature. This distinction matters because it determines how requests are routed, which approvals are required, and how the student's status display reflects the position of the request in the chain.

There is also a gap in terms of deployment approach. Most systems reviewed in the literature either require installation as a mobile application or depend on an institution-specific server setup, both of which create barriers to adoption. A fully web-based, cloud-deployed solution that requires no installation and works on any modern browser addresses a practical constraint that is particularly relevant for institutions with limited IT infrastructure.

GateVault addresses these identified gaps by providing a complete, end-to-end gate pass management system with a five-role access hierarchy, a dynamic QR verification module, real-time status tracking, and cloud-based deployment through Vercel — all within a single, mobile-responsive web application.

Chapter 3

Requirement Analysis

Requirement analysis is the foundation on which any software system is designed and built. It involves identifying what the system must do, under what conditions it must perform, what resources are needed to develop and run it, and whether the project is viable given the constraints at hand. This chapter documents the requirement analysis carried out for the Advanced Gate Pass System, covering the development methodology, functional and non-functional requirements, hardware and software specifications, and a feasibility assessment.

3.1 Development Methodology

The project was developed following an Incremental Development Model, which is a structured approach in which the system is built and delivered in functional stages rather than as a single complete release. Each stage, referred to as an increment, adds a defined set of features to the existing working system. This approach was chosen because the system is composed of five clearly distinct user modules — Student, HoD, Warden, Security Guard, and Administrator — each of which could be designed, built, and tested independently before being integrated into the whole.

The development proceeded through the following increments. In the first increment, the core infrastructure was established — the project was set up using Next.js, the MongoDB database was configured, and the authentication system using NextAuth.js was implemented. In the second increment, the Student module was built, covering registration, login, form submission, and status tracking. The third increment introduced the HoD and Warden modules, along with the approval routing logic that determines how a request is directed based on its type and date. The fourth increment implemented the QR code generation system on the approval side and the QR scanning interface for the Security Guard module. The fifth increment completed the Admin dashboard and history logging features across all roles. A final round of integration testing, bug fixing, and UI refinement was carried out before deployment.

This incremental approach offered the advantage of early testing at each stage, allowing the team to identify design issues and correct them before they propagated into subsequent modules. It also allowed the system to remain functional and demonstrable throughout the development period, which was useful for periodic reviews with the project supervisors.

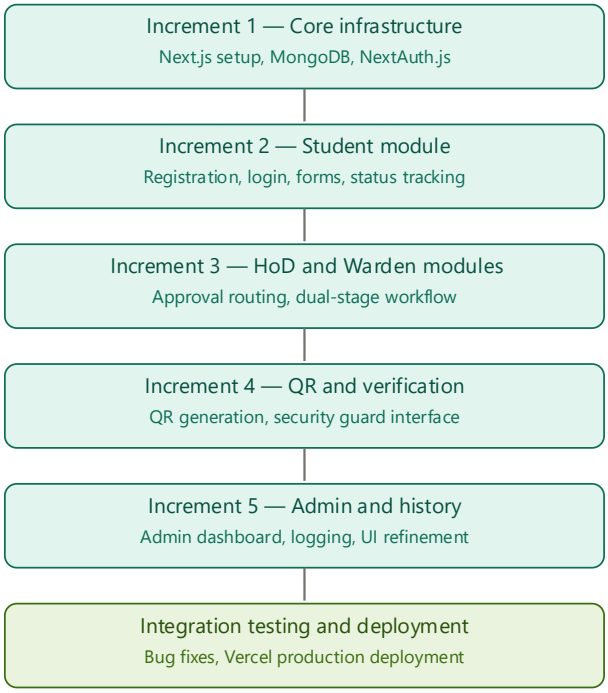

Figure 3.1: Incremental Development Model — Stages of GateVault Development

3.2 Functional Requirements

Functional requirements define the specific actions and behaviours the system must be capable of performing. The following requirements were identified through an analysis of the manual process in place and through consultation with the project supervisors.

3.2.1 User Registration and Authentication

The system must allow users to register with their institutional details and log in securely using email and password credentials. Passwords must be stored in encrypted form. Each user account must be associated with a specific role, and access to any part of the system must be restricted to authenticated users only.

3.2.2 Student Module

The system must allow a logged-in student to submit two types of requests — a Gate Pass form for short-duration campus exits and a Leave form for overnight or multi-day absence. Each submission must receive a unique Request ID automatically. The student must be able to view the current approval status of any pending request in real time from their dashboard. Approved requests must generate a temporary QR code that the student can present at the gate. The

student must also be able to view a complete history of all past requests along with their outcomes.

3.2.3 HoD Module

The system must route Leave requests submitted on working days to the HoD for the first stage of approval. The HoD must be able to view all incoming requests assigned to them, approve or deny each request individually, search for requests by student name, and view a full log of past decisions. Only leave requests must appear in the HoD queue — gate pass requests must not be routed to the HoD at any stage.

3.2.4 Warden Module

The system must route all Gate Pass requests directly to the Warden. Leave requests must appear in the Warden queue only after the HoD has approved them. The Warden must be able to approve or deny requests, search by student name, and view a history log. Upon the Warden's approval, the system must trigger QR code generation for the student's request.

3.2.5 Security Guard Module

The system must provide the Security Guard with a QR scanning interface through which they can scan a student's QR code and immediately view the approval status of the associated request. The guard must also be able to see a log of all previously scanned requests during their session.

3.2.6 Admin Module

The Administrator must have a dashboard that provides a complete view of all requests across all students and roles. The Admin must be able to search by student name, view status lists, and access full history logs. The Admin does not participate in the approval workflow but has read access to all system activity.

3.2.7 Request Routing Logic

The system must automatically determine the approval path for each submitted request based on its type. A Gate Pass request must be sent to the Warden only. A Leave request submitted on a working day must be sent first to the HoD and then, upon HoD approval, forwarded to the Warden. A Leave request submitted for a holiday period must follow the routing as defined during configuration.

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints under which the system must operate. They do not describe specific functions but determine how well those functions are carried out.

Performance — The system must load dashboard pages within an acceptable time frame under normal network conditions. API responses for approval actions and QR code generation must complete within two seconds to ensure that the workflow does not feel sluggish, particularly during high-submission periods.

Security — User passwords must be hashed using bcryptjs before storage. Session management must be handled through NextAuth.js with secure token-based sessions. No sensitive data must be exposed through client-side code or publicly accessible API endpoints. Role boundaries must be enforced at the server side so that a user cannot access another role's data by manipulating the frontend.

Usability — The interface must be intuitive enough that a first-time user can navigate their dashboard and complete a primary task — such as submitting a gate pass or approving a request — without requiring any training. All forms must include appropriate input validation with clear error messages.

Responsiveness — The application must be fully functional on mobile devices, tablets, and desktop browsers without any loss of features. Since most students and wardens are likely to access the system from a smartphone, the mobile layout must be given equal design priority as the desktop layout.

Availability — The system must maintain high availability during peak submission periods, such as the start of semester breaks. Deployment through Vercel provides automatic scaling and uptime management that supports this requirement.

Maintainability — The codebase must be structured using clearly separated components and modules so that future developers can update individual parts of the system without disrupting unrelated functionality. TypeScript is used throughout to enforce type safety and reduce runtime errors.

3.4 Hardware and Software Requirements

3.4.1 Hardware Requirements

The system is web-based and requires no dedicated server hardware, as it is deployed on Vercel's cloud infrastructure. From the user's perspective, the minimum hardware requirement is any device capable of running a modern web browser — this includes smartphones, tablets, and desktop or laptop computers. For the Security Guard's QR scanning functionality, the device must have a functional camera. No additional hardware is required at the institution's end.

Table 3.1: Minimum Client Hardware Requirements

Sl. No.	Component	Minimum Specification	Remarks
1	Device Type	Smartphone, tablet, or desktop / laptop computer	Any internet-capable device with a modern web browser is sufficient.
2	Processor (CPU)	Any dual-core processor, 1 GHz or above	No intensive computation is performed client-side; the browser handles all rendering.
3	RAM (Memory)	Minimum 1 GB RAM	2 GB or above is recommended for a smoother browser experience, especially on mobile devices.
4	Storage	Minimum 100 MB free storage	Required only for browser cache and temporary data. No application installation is needed.
5	Display / Screen	Minimum screen width of 360 px	The application uses a mobile-first responsive layout; all dashboards scale correctly from 360 px upward.
6	Camera	Minimum 5 MP rear or front camera	Mandatory for the Security Guard module only. QR code scanning is performed via the browser's camera API.
7	Internet Connection	Active connection at minimum 1 Mbps (Wi-Fi or mobile data)	A stable connection is required at all times, particularly during QR verification, which queries the live database.
8	Operating System	Windows 8+, macOS 10.13+, iOS 14+, or Android 8.0+	Any OS that supports a modern, standards-compliant web browser is compatible.
9	Web Browser	Google Chrome 90+, Mozilla Firefox 88+, Apple Safari 14+, or Microsoft Edge 90+	JavaScript must be enabled. Internet Explorer is not supported.

Note — No additional hardware installation is required. GateVault is entirely browser-based and is deployed on Vercel's cloud infrastructure.

3.4.2 Software Requirements

The development environment and the software tools used to build and deploy the system are listed below.

Table 3.2: Software and Tools Used

Sl. No.	Category	Tool / Technology	Version	Purpose in the Project
FRONTEND				
1	Full-Stack Framework	Next.js	14.x	Serves as the unified frontend and backend framework. The App Router handles page routing, and API Routes serve all backend endpoints.
2	UI Library	React	18.x	Provides the component-based architecture for all role-specific dashboards. Hooks manage state and side effects across the application.
3	Programming Language	TypeScript	5.x	Enforces static type checking throughout the codebase, reducing runtime errors and improving code maintainability across all modules.
4	Styling Framework	Tailwind CSS	3.x	Handles all UI styling via utility classes. Its mobile-first breakpoint system ensures the application is fully responsive across device sizes.
5	Animation Library	Framer Motion	10.x	Adds page transition animations and component-level micro-interactions such as card slide-ins and status badge fade effects.
6	Icon Library	Lucide React	Latest	Provides consistent, lightweight SVG icons used throughout the navigation, forms, and dashboard interfaces.
7	Data Visualisation	Recharts	2.x	Renders bar charts in the Admin and Warden dashboards displaying request count summaries by status category.
8	QR Code Generation	QRCode React	Latest	Generates scannable QR code images from the approved Request ID string, displayed on the student dashboard upon approval.
9	QR Code Scanning	HTML5 QR Code	Latest	Powers the camera-based QR scanning interface on the Security Guard dashboard using the browser's native media API.

BACKEND AND AUTHENTICATION				
10	Authentication	NextAuth.js	4.x	Manages session creation, JWT token validation, and secure cookie handling. Integrates with the Credentials Provider for email-password login.
11	Password Security	bcryptjs	Latest	Hashes user passwords before storage in the database. Ensures that plaintext credentials are never written to MongoDB at any point.
DATABASE				
12	Database	MongoDB Atlas	6.x	Cloud-hosted NoSQL database storing Users, Requests, and Scans collections. Provides built-in replication, automatic backups, and a free-tier sufficient for single-institution usage.
13	Object Data Modelling	Mongoose	7.x	Defines schemas for each MongoDB collection and enforces data types, required fields, and enumerated constraints at the application level.
DEVELOPMENT AND VERSION CONTROL				
14	Code Quality	ESLint	8.x	Performs static code analysis during development to catch syntax errors, unused variables, and style inconsistencies before runtime.
15	Package Manager	npm	10.x	Manages all project dependencies. The package.json file defines all libraries and their required version ranges for reproducible builds.
16	Version Control	Git and GitHub	—	Tracks all code changes incrementally throughout development. The public repository is hosted at GitHub and connected to the Vercel deployment pipeline.
DEPLOYMENT				
17	Cloud Deployment	Vercel	—	Hosts the entire Next.js application — frontend and API routes — as a single serverless deployment. Every push to the main branch triggers an automated build and production release.

Note — All tools listed above are open-source and freely available. No proprietary software licences were required at any stage of development.

3.5 Feasibility Study

A feasibility study was conducted prior to the commencement of development to assess whether the project was viable from technical, operational, and economic standpoints.

3.5.1 Technical Feasibility

All technologies selected for the project are mature, well-documented, and widely used in production applications. Next.js, React, and MongoDB are supported by large open-source communities and have established best practices for the kind of multi-role web application being built. The team had prior exposure to JavaScript and web development concepts, and the incremental methodology allowed technical knowledge gaps to be addressed progressively as each module was developed. Deployment through Vercel eliminates the need for server provisioning and configuration, making the infrastructure side of the project manageable within the team's capabilities. The project is, therefore, technically feasible.

3.5.2 Operational Feasibility

The system is designed to replace a process that users already perform regularly — submitting and approving gate pass forms. The digital version follows the same logical steps as the manual process, which means that the learning curve for end users is minimal. Students, wardens, and security guards interact with straightforward dashboards that require no specialised technical knowledge. Since the system is browser-based, no installation is needed on any device. The operational transition from a manual to a digital workflow is therefore feasible without requiring any significant change in how the institution is administered.

3.5.3 Economic Feasibility

The entire system was developed using open-source technologies, all of which are freely available without licensing costs. Hosting on Vercel is available under a free tier that is sufficient for the scale of a single institution's usage. The only recurring cost that an institution would incur upon formal adoption is a MongoDB Atlas subscription if usage grows beyond the free tier, which provides 512 MB of storage — adequate for a small to medium institution's gate pass data over a full academic year. The project involves no hardware procurement, no proprietary software licences, and no ongoing maintenance infrastructure costs at the development stage. The system is, therefore, economically feasible for both development and deployment.

Chapter 4

System Design and Architecture

System design translates the requirements identified in the previous chapter into a concrete blueprint that guides development. This chapter documents the architectural decisions made for GateVault, describes the end-to-end workflow of the system, and presents the key design artefacts — the Data Flow Diagram, Use Case Diagram, and Entity Relationship Diagram — that formally represent the structure and behaviour of the application.

4.1 System Architecture

GateVault is built on a three-tier client-server architecture comprising a Presentation Layer, an Application Layer, and a Data Layer. This separation of concerns ensures that each part of the system has a clearly defined responsibility, making the codebase easier to maintain and extend.

4.1.1 Presentation Layer

The Presentation Layer is the part of the system that the user directly interacts with. It is built using React and Next.js, with Tailwind CSS managing the responsive layout and styling. Each user role has a dedicated dashboard that is rendered on the client side, with state managed within the React component tree. Framer Motion is used for transitions and micro-interactions, and Recharts is integrated in the Admin and Warden dashboards for visual summaries of request data. The QR code display for students is handled by the QRCode React library, while the camera-based scanning interface for security guards uses the HTML5 QR Code library.

4.1.2 Application Layer

The Application Layer handles all business logic and data operations. Next.js API Routes serve as the backend, processing incoming requests from the frontend, enforcing role-based access control, executing approval logic, and communicating with the database. Authentication is managed through NextAuth.js, which handles session creation, token validation, and secure cookie management. Passwords are hashed using bcryptjs before being stored, ensuring that plaintext credentials are never written to the database. All API endpoints validate the user's session and role before executing any operation, meaning that access restrictions are enforced on the server side rather than relying solely on frontend routing.

4.1.3 Data Layer

The Data Layer consists of a MongoDB database hosted on MongoDB Atlas, Mongoose is used as the Object Data Modelling library to define schemas and interact with the database in a structured way. Atlas provides cloud-based hosting with built-in replication and automatic backups, removing the need for the team to manage any database infrastructure directly.

The overall deployment is handled by Vercel, which hosts both the frontend and the Next.js API backend as a single unified deployment. The client's browser communicates with the Vercel-hosted application through HTTPS, and the application in turn communicates with MongoDB Atlas over a secure connection string.

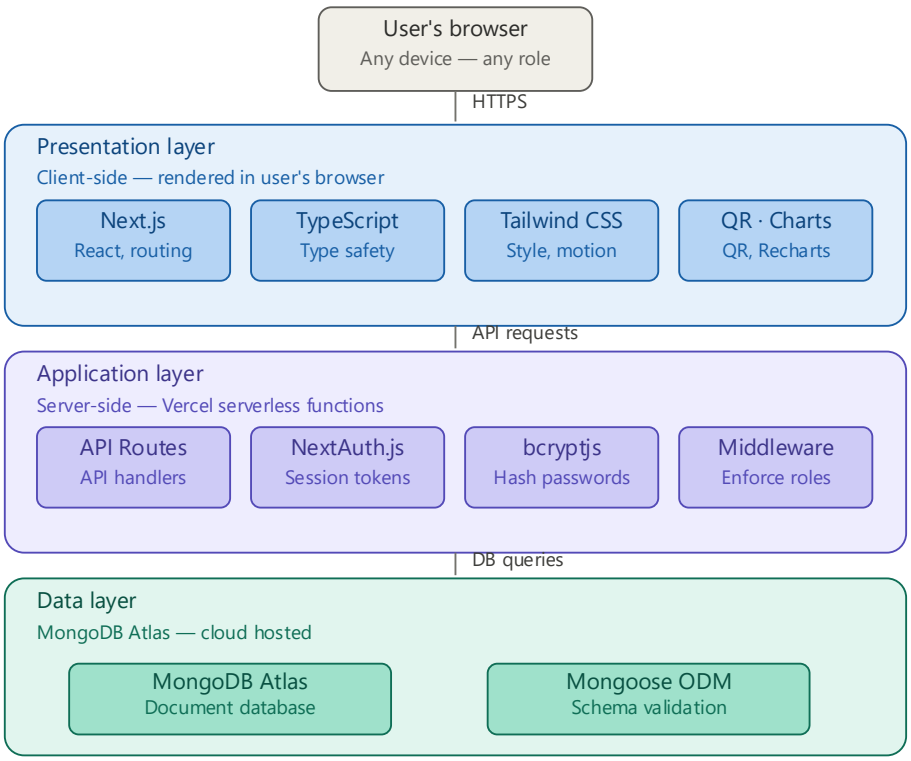

Figure 4.1: Three-Tier System Architecture Diagram of GateVault

4.2 Workflow of the System

The system workflow describes the sequence of events that take place from the moment a student initiates a request to the point at which they exit the campus gate. There are two distinct request types — Gate Pass and Leave — each of which follows a different approval path.

4.2.1 Gate Pass Workflow

A Gate Pass is intended for short-duration exits, typically within the same day. When a student submits a Gate Pass request, the system assigns a unique Request ID and immediately routes the request to the Hostel Warden. The Warden reviews the request from their dashboard and either approves or denies it. If denied, the request is closed and the student is notified through their status dashboard. If approved, the system generates a temporary QR code linked to the specific Request ID and makes it available on the student's dashboard. The student presents this QR code at the campus gate. The Security Guard scans the code using their dashboard interface, which queries the database and returns the current status of the associated request. The student is permitted to exit if the status is Approved.

4.2.2 Leave Request Workflow

A Leave request covers overnight or multi-day absences. When a student submits a Leave request on a working day, the request is routed to the HoD as the first approving authority. The HoD reviews and either approves or denies it. A denial at this stage closes the request. Upon HoD approval, the system automatically forwards the request to the Hostel Warden for the second stage of approval. The Warden then makes the final decision. A QR code is generated only after the Warden's approval, following the same gate verification process described above.

4.2.3 Return Process

Upon returning to the campus, the student's return is recorded by the Warden or Admin by updating the request status in the system. This creates a complete movement record for that request in the database.

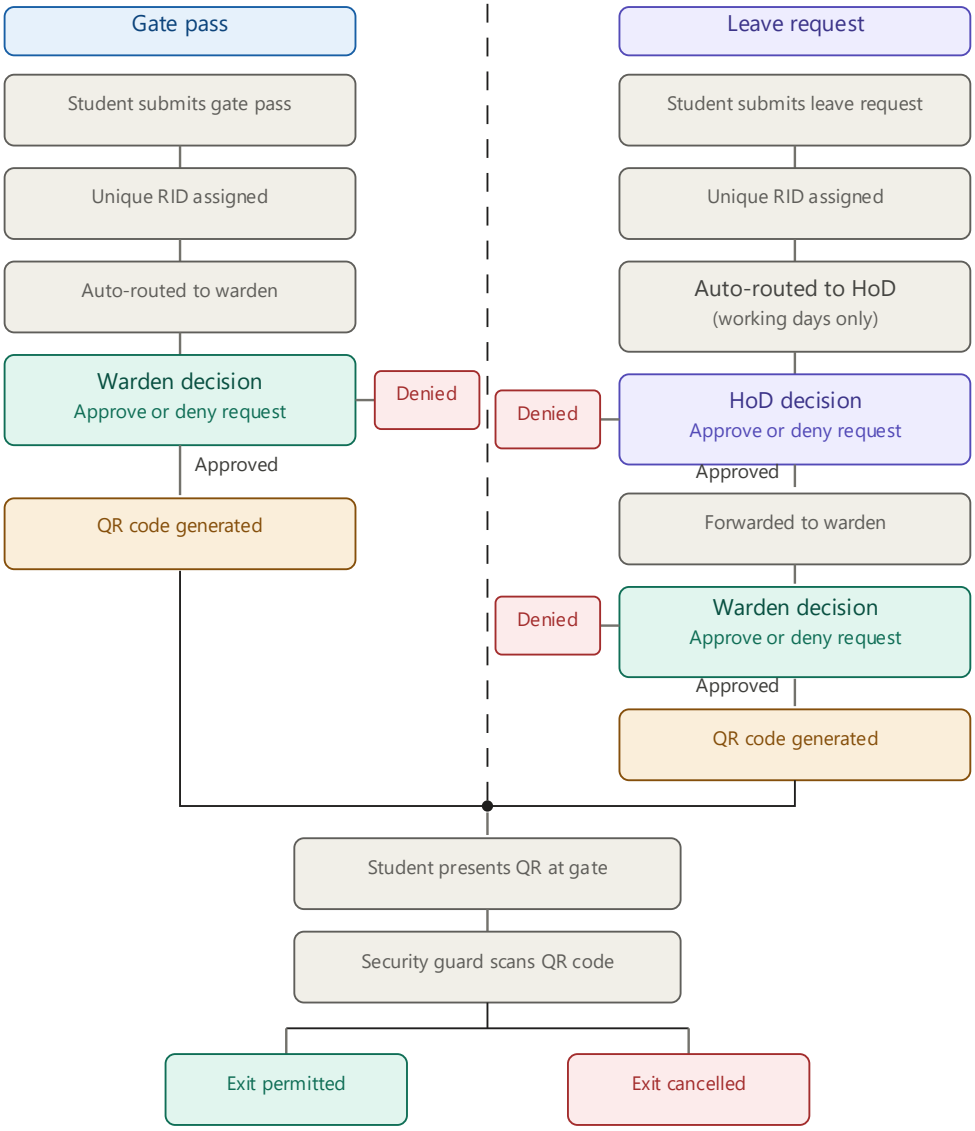

Figure 4.2: End-to-End Workflow Diagram — Gate Pass and Leave Request Paths

4.3 Data Flow Diagram (DFD)

A Data Flow Diagram illustrates how data moves through a system, identifying the processes, data stores, external entities, and the flows that connect them. Two levels of DFD are presented here — a Level 0 context diagram and a Level 1 detailed diagram.

4.3.1 Level 0 — Context Diagram

The Level 0 DFD presents the system as a single process and shows its interaction with the five external entities: Student, HoD, Hostel Warden, Security Guard, and Administrator. The Student sends request data into the system and receives status information and a QR code in return. The HoD and Warden each send approval or denial decisions into the system and receive

incoming request data from it. The Security Guard sends scan inputs and receives verification results. The Administrator interacts with the system to view records and manage user data. All interactions pass through the central GateVault system process.

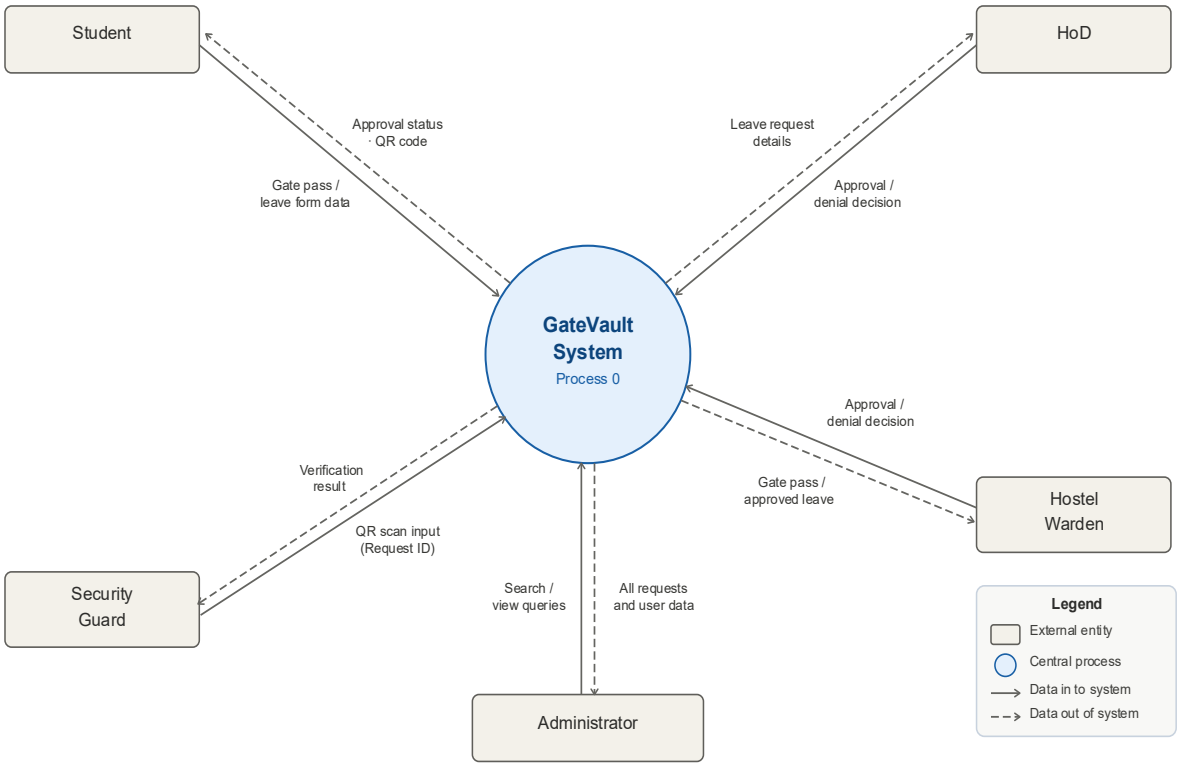

Figure 4.3: Level 0 DFD — Context Diagram

4.3.2 Level 1 — Detailed DFD

The Level 1 DFD breaks the central system process into its major sub-processes: User Authentication, Request Submission, Approval Processing, QR Code Generation, Gate Verification, and Record Management. Data flows from the Student entity into the Request Submission process, which writes to the Requests data store and triggers the Approval Processing sub-process. The Approval Processing sub-process reads from the Requests data store, receives input from the HoD and Warden entities, and updates the request status accordingly. On approval, the QR Code Generation sub-process reads the approved request from the data store and returns the generated code to the Student. The Gate Verification sub-process receives scan input from the Security Guard entity, reads the request status from the data store, and returns a verification result. The Record Management sub-process serves the Administrator by reading all entries from the Requests and Users data stores.

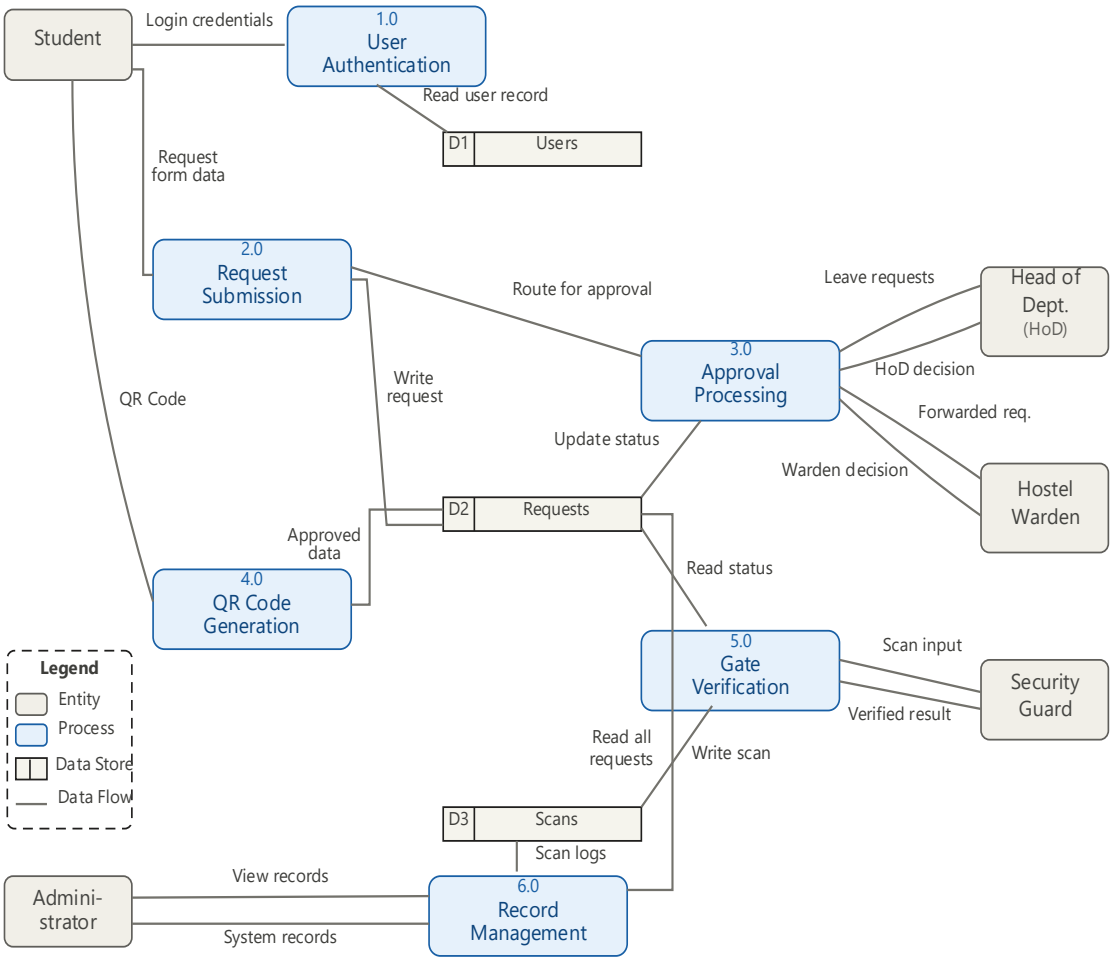

Figure 4.4: Level 1 DFD — Detailed Process Diagram

4.4 Use Case Diagram

The Use Case Diagram identifies the actors in the system and the specific actions each actor can perform. The five actors are Student, HoD, Hostel Warden, Security Guard, and Administrator, each of whom interacts with a defined set of use cases within the system boundary.

The Student actor is associated with the following use cases: Register and Login, Submit Gate Pass Request, Submit Leave Request, View Request Status, View QR Code, and View Request History.

The HoD actor is associated with: Login, View Incoming Leave Requests, Approve Request, Deny Request, Search by Student Name, and View Approval History.

The Hostel Warden actor is associated with: Login, View Incoming Gate Pass Requests, View Forwarded Leave Requests, Approve Request, Deny Request, Search by Student Name, and View Approval History.

The Security Guard actor is associated with: Login, Scan QR Code, View Verification Result, and View Scan History.

The Administrator actor is associated with: Login, View All Requests, Search by Student Name, View Status List, and View Full System History.

Several use cases, such as Login and View History, appear across multiple actors but are treated as separate instances since they operate on role-specific data and interfaces.

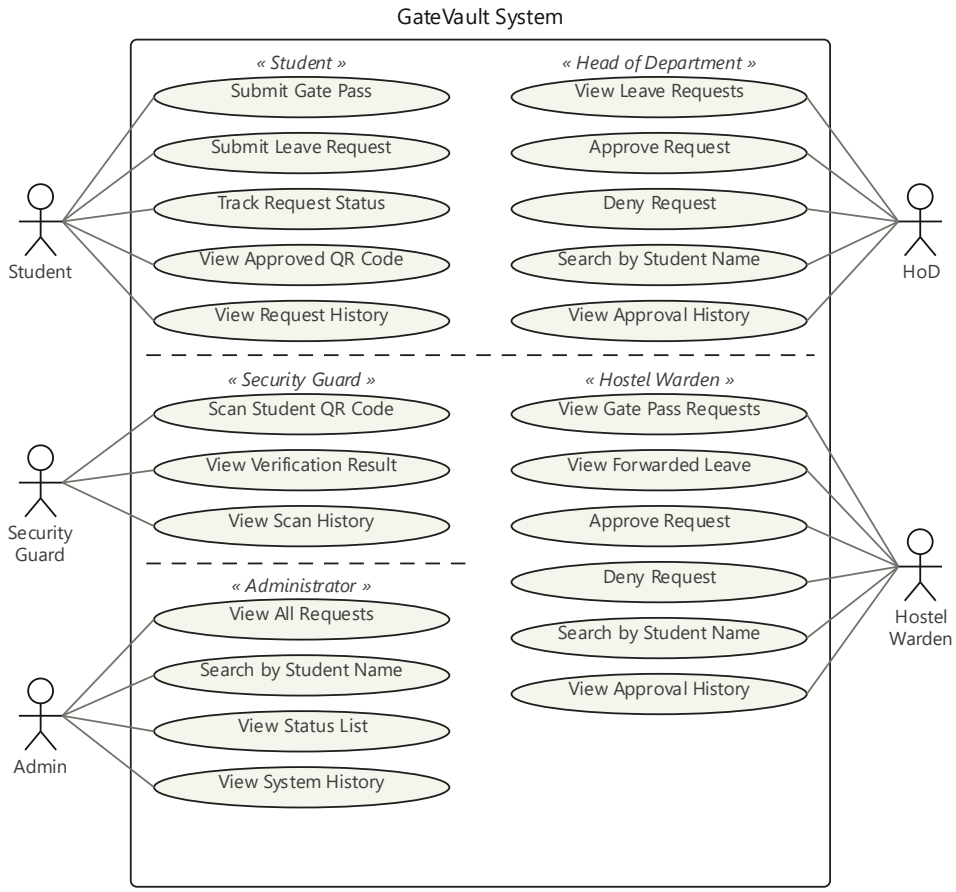

Figure 4.5: Use Case Diagram — All Actors and System Interactions

4.5 ER Diagram and Database Design

MongoDB is a document-oriented database that stores data in flexible JSON-like documents rather than in fixed relational tables. Despite this flexibility, the data in GateVault is structured

around three primary collections — Users, Requests, and Scans — which have defined relationships between them enforced at the application level through Mongoose schemas.

4.5.1 Users Collection

The Users collection stores the account information for every registered user, regardless of role. The role field is used throughout the application to determine what data each user can access and what actions they can perform.

Table 4.1: Users Collection Schema

Database: MongoDB Atlas · **Collection:** `users`

Sl. No.	Field Name	Data Type	Constraints	Description
1	_id	ObjectId	Auto-generated, Primary Key	Unique document identifier generated automatically by MongoDB for every new user record.
2	name	String	Required	Full name of the registered user. Stored as a trimmed string and displayed across all dashboard interfaces.
3	email	String	Required, Unique	Institutional email address used as the login identifier. Stored in lowercase. Duplicate email addresses are not permitted across any role.
4	password	String	Required, bcryptjs hashed	User password stored exclusively in bcryptjs-hashed form. Plaintext passwords are never written to the database at any point.
5	role	String	Required, Enum	Determines the user's access level and dashboard. Accepted values: student, hod, warden, security, admin
6	studentId	String	Optional	Institutional roll number of the student (e.g., ET22BTHCS072). Populated only for accounts with role = student.
7	department	String	Optional	Name of the academic department to which the user belongs. Used for display purposes and for associating students with the correct HoD.
8	hostelRoom	String	Optional	Hostel room number assigned to the student (e.g., B-313). Populated only for accounts with role = student.
9	createdAt	Date	Auto-generated	Timestamp recording the exact date and time the user account was created. Defaults to Date.now and is set automatically by Mongoose on document creation.

Required — Field must be present on every document

Unique — Value cannot be duplicated across documents

Optional — Populated conditionally based on role

Enum — Value restricted to a defined set

4.5.2 Requests Collection

The Requests collection is the central data store of the system. Every gate pass and leave submission creates a document in this collection. The document is updated as the request moves through the approval workflow.

Table 4.2: Requests Collection Schema

Database: MongoDB Atlas · **Collection:** `requests`

Sl. No.	Field Name	Data Type	Constraints	Description
DOCUMENT IDENTITY				
1	_id	ObjectId	**Auto-generated** **PrimaryKey**	Unique document identifier generated automatically by MongoDB on creation of every new request record.
2	requestId	String	**Required** **Unique**	Human-readable Request ID (RID) assigned at submission — for example, RID-20260316-0042. Used for QR code encoding and student-facing display.
STUDENT INFORMATION				
3	studentId	ObjectId	**Required** **Ref: users**	Reference to the _id of the student who submitted the request in the Users collection. Enables population of full student details via Mongoose.
4	studentName	String	**Required**	Full name of the student captured at the time of submission. Stored directly in the request document to avoid lookup overhead during approval and search operations.

REQUEST DETAILS				
5	type	String	Required Enum gate_pass leave	Determines the approval routing path. A gate_pass is routed directly to the Warden. A leave request on a working day is routed to the HoD first.
6	reason	String	Required	The reason for the request as stated by the student at the time of form submission. Displayed to the HoD and Warden for review.
7	destination	String	Required	The intended location to which the student plans to travel. Provides contextual information for the approving authority.
8	fromDate	Date	Required	The date and time from which the student intends to be absent from campus. Also used by the routing logic to determine whether HoD approval is required.
9	toDate	Date	Required	The expected date and time of the student's return to campus. Recorded for institutional movement history and duration tracking purposes.
APPROVAL WORKFLOW				
10	hodStatus	String	Required Enum Pending Approved Denied not_required	Records the HoD's decision on the request. Set to not_required automatic ally for Gate Pass requests and for Leave requests submitted on non-working days. Defaults to pending.
11	wardenStatus	String	Required Enum Pending Approved denied	Records the Warden's final decision on the request. A Warden approval on a Leave request is only permitted after hodStatus has been set

				to approved or not_required. Defaults to pending.
12	hodApprovedBy	ObjectId	**Optional** **Ref: users**	Reference to the _id of the HoD who approved or denied the request. Populated only when an HoD action has been taken. Null for Gate Pass requests.
13	wardenApprovedBy	ObjectId	**Optional** **Ref: users**	Reference to the _id of the Warden who took the final approval decision. Populated only after Warden action is recorded in the workflow.
14	overallStatus	String	**Required** **Enum** **Pending** **Approved** **denied**	The consolidated status of the request derived from both individual stage statuses. This is the field the student dashboard and the QR verification API read at runtime. Defaults to pending.
QR CODE AND TIMESTAMPS				
15	qrCodeData	String	**Optional**	The encoded string written into the QR code image — typically the requestId value. Populated only upon full approval by the Warden. Null for all pending and denied requests.
16	createdAt	Date	**Auto-generated**	Timestamp recorded automatically by Mongoose at the moment the student submits the request. Used for sorting and chronological display in history logs.
17	updatedAt	Date	**Auto-generated**	Timestamp updated automatically by Mongoose each time the request document is modified — for example, when an approval or denial is recorded at any stage of the workflow.

Required — Must be present **Unique** — No duplicates permitted

Optional — Conditionally populated **Ref: users** — Foreign key reference to Users collection

4.5.3 Scans Collection

The Scans collection records every QR code scan performed by a security guard, creating a verifiable log of gate exit events.

Table 4.3: Scans Collection Schema

Database: MongoDB Atlas · **Collection:** `scans`

Sl. No.	Field Name	Data Type	Constraints	Description
1	_id	ObjectId	**Auto-generated** **Primary Key**	Unique document identifier generated automatically by MongoDB for every new scan event recorded by the Security Guard.
2	requestId	ObjectId	**Required** **Ref: requests**	Reference to the _id of the associated request document in the Requests collection. Links each scan event to the specific gate pass or leave request that was presented at the gate.
3	scannedBy	ObjectId	**Required** **Ref: users**	Reference to the _id of the Security Guard who performed the scan, sourced from the Users collection. Provides a complete audit trail associating each verification event with the guard on duty.
4	statusAtScan	String	**Required** **Enum** **Pending** **Approved** **denied**	The overallStatus of the associated request at the exact moment the QR code was scanned. Stored as a snapshot rather than a live reference, ensuring the historical record remains accurate even if the request status changes after the scan event.
5	scannedAt	Date	**Auto-generated** **Default: Date.now**	Precise timestamp of the moment the Security Guard performed the QR scan. Recorded automatically via Date.now and displayed in the guard's session history log.

Collection Relationships

scans.requestId → **requests._id** Many-to-one — multiple scans may reference the same approved request.

scans.scannedBy → **users._id** Many-to-one — a single guard account may perform many scans over time.

Required — Field must be present on every document

Enum — Value restricted to a predefined set

Ref: collection — Foreign key reference to another collection

4.5.4 Entity Relationships

A one-to-many relationship exists between the Users collection and the Requests collection, since a single student can submit multiple requests over time. A one-to-many relationship also exists between the Requests collection and the Scans collection, as the same approved request could be scanned more than once — for example, upon exit and again if the guard needs to re-verify. References between documents are stored as ObjectId fields, which Mongoose resolves through population queries when related data needs to be fetched together.

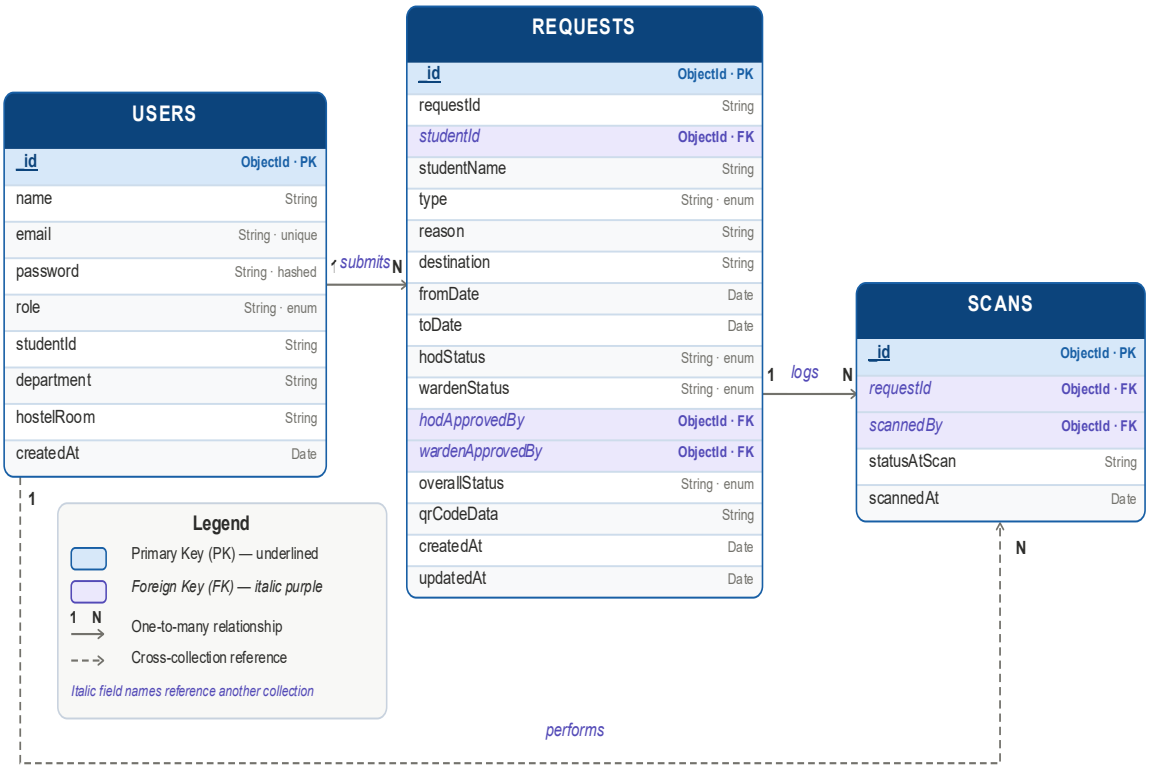

Figure 4.6: Entity Relationship Diagram — Users, Requests, and Scans Collections

Chapter 5

System Implementation

This chapter documents how the design specifications from Chapter 4 were translated into a working application. It covers the frontend and backend implementation, the authentication and authorisation mechanism, the QR code generation and scanning modules, and the database integration and deployment process.

5.1 Frontend Implementation

The frontend of GateVault is built using Next.js 14 with the App Router, which provides a file-system-based routing structure where each folder within the /app directory corresponds to a URL path. React is used as the underlying UI library, with all components written in TypeScript to enforce strict type checking across the codebase.

5.1.1 Project Structure

The application is organised around role-specific route segments. Each user role has its own dashboard directory under `/app/dashboard/`, ensuring that the code for each module remains self-contained and maintainable. Shared components such as navigation bars, form elements, status badges, and modal dialogs are placed in a `/components` directory and reused across modules where applicable.

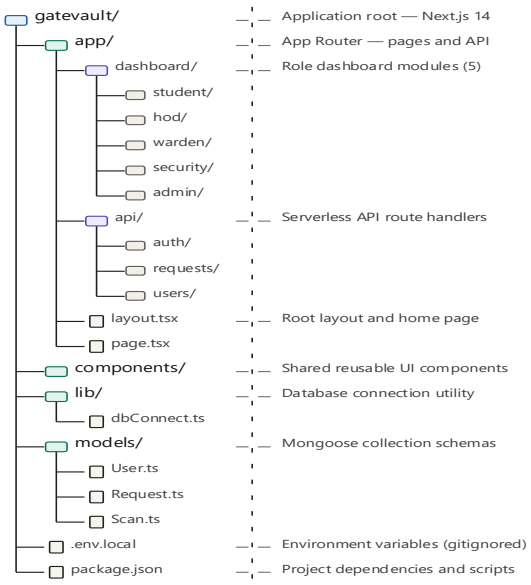

Figure 5.1: Project Directory Structure of GateVault

5.1.2 Responsive Design with Tailwind CSS

Tailwind CSS is used for all styling through utility classes applied directly in the component markup. Its mobile-first breakpoint system was used consistently throughout — styles are written for small screens by default and progressively overridden for larger viewports using md: and lg: prefixes. This approach ensured that every dashboard remained fully functional on mobile devices without requiring a separate mobile layout to be maintained.

5.1.3 Animations and Visual Feedback

Framer Motion is used to add page transition animations and component-level micro-interactions such as card slide-ins and status badge fade effects. These animations improve the perceived responsiveness of the interface and help direct the user's attention during state changes — for example, when a request status updates from Pending to Approved.

5.1.4 Data Visualisation

The Admin and Warden dashboards include summary charts built using the Recharts library. A bar chart displays the count of requests by status — Pending, Approved, and Denied — for the current period, giving authorities a quick visual overview of workload without having to read through individual records.

5.1.5 Role-Based Dashboard Components

Each dashboard is composed of functional React components that fetch data from the backend API using the `useEffect` hook on mount and update local component state on response. The Student dashboard includes components for form submission, status display, QR code viewing, and history logs. The HoD and Warden dashboards include an incoming requests list, an action panel for approve and deny decisions, and a search bar. The Security Guard dashboard contains the QR scanning interface and a scanned requests log. The Admin dashboard provides a full-system overview with search and filtering capabilities.

```typescript
// Example: Fetching student requests on dashboard load
useEffect(() => {
  const fetchRequests = async () => {
    const res = await fetch('/api/requests/student');
    const data = await res.json();
    setRequests(data.requests);
  };
  fetchRequests();
}, []);
```

5.2 Backend Implementation

The backend of GateVault is implemented entirely within Next.js using API Routes, which are serverless functions that handle HTTP requests. Each route file within the `/app/api/` directory exports handler functions for the HTTP methods it supports — typically GET for data retrieval and POST or PATCH for submissions and updates.

5.2.1 API Route Organisation

API routes are organised by resource. The `/api/requests` route handles submission of new gate pass and leave forms. The `/api/requests/[id]` route handles approval, denial, and status updates for a specific request. The `/api/users` route handles user management operations accessible to the Admin. All routes check the user's session and role before executing any database operation, as described in Section 5.3.

5.2.2 Request Routing Logic

One of the more important pieces of backend logic is the function that determines the approval path for a submitted request. When a new request document is created in the database, this function examines the request type and the submission date, then sets the initial `hodStatus` and `wardenStatus` fields accordingly.

```typescript
// Approval path logic on request submission
const isWorkingDay = (date: Date): boolean => {
  const day = date.getDay();
  return day !== 0 && day !== 6;
};

if (type === 'gate_pass') {
  hodStatus = 'not_required';
  wardenStatus = 'pending';
} else if (type === 'leave' && isWorkingDay(new Date(fromDate))) {
  hodStatus = 'pending';
  wardenStatus = 'pending';
} else {
  hodStatus = 'not_required';
  wardenStatus = 'pending';
}
```

When a Warden approves a request, a separate function checks whether the `hodStatus` is either approved or `not_required` before allowing the overall status to be set to approved. This ensures that the two-stage approval chain for leave requests on working days cannot be bypassed.

5.2.3 Middleware for Role Enforcement

A custom middleware function is applied to all protected API routes. It reads the session token from the incoming request, verifies it using NextAuth.js, and checks whether the authenticated user's role matches the role permitted to access that endpoint. If the check fails, the route returns a 403 Forbidden response immediately without executing any database operation.

```typescript
// Role guard middleware
export const requireRole = (allowedRoles: string[]) => {
  return async (req: NextRequest) => {
    const session = await getServerSession(authOptions);
    if (!session || !allowedRoles.includes(session.user.role)) {
      return NextResponse.json(
        { error: 'Access denied' },
        { status: 403 }
      );
    }
  };
};
```

5.3 Authentication and Authorization

Authentication is handled through NextAuth.js using the Credentials Provider, which allows users to log in with an email address and password. This approach was chosen over third-party OAuth providers because the system is intended for internal institutional use, where students and staff are registered with institutional email addresses rather than personal Google or GitHub accounts.

5.3.1 NextAuth Configuration

The NextAuth configuration is defined in `/app/api/auth/[...nextauth]/route.ts`. The Credentials Provider receives the submitted email and password, queries the Users collection in MongoDB for a matching email, and uses bcryptjs to compare the submitted password against the stored hash. If authentication is successful, the user's role and ID are added to the session token so that they are available throughout the application without requiring additional database queries.

```typescript
// NextAuth credentials provider
CredentialsProvider({
  name: 'Credentials',
  credentials: {
    email: { label: 'Email', type: 'email' },
    password: { label: 'Password', type: 'password' },
  },
  async authorize(credentials) {
    const user = await User.findOne({ email: credentials?.email });
    if (!user) return null;
    const isValid = await bcrypt.compare(
      credentials!.password,
      user.password
    );
    if (!isValid) return null;
    return {
      id: user._id.toString(),
      name: user.name,
      email: user.email,
      role: user.role,
    };
  },
})
```

5.3.2 Session Strategy

NextAuth is configured to use JSON Web Tokens (JWT) for session management. The JWT callback is used to persist the user's role and database ID into the token at login, and the session callback exposes these fields on the `session.user` object, making them accessible in both server-side and client-side code throughout the application.

```typescript
// JWT and session callbacks
callbacks: {
  async jwt({ token, user }) {
    if (user) {
      token.role = user.role;
      token.id = user.id;
    }
    return token;
  },
  async session({ session, token }) {
    session.user.role = token.role;
    session.user.id = token.id;
    return session;
  },
}
```

5.3.3 Frontend Route Protection

On the client side, each dashboard page checks for an active session using the `useSession` hook from NextAuth.js. If no session exists, the user is redirected to the login page. If a session exists but the user's role does not match the dashboard they are attempting to access, they are redirected to their own role-appropriate dashboard. This prevents a student from manually navigating to the warden's dashboard URL.

5.4 QR Code Generation and Verification

The QR code module is one of the most functionally significant parts of the system, as it forms the bridge between the digital approval workflow and the physical gate verification process.

5.4.1 QR Code Generation

When a request receives full approval — either from the Warden alone in the case of a Gate Pass, or from both the HoD and Warden in the case of a Leave request on a working day — a QR code string is constructed and stored in the `qrCodeData` field of the request document. The QR string encodes the unique Request ID (RID) of the approved request. On the student's dashboard, the `QRCode` component from the QRCode React library renders this string as a scannable QR image.

```typescript
// QR code display on student dashboard
import QRCode from 'react-qr-code';

{request.overallStatus === 'approved' && (
  <div className="flex flex-col items-center gap-4">
    <QRCode
      value={request.requestId}
      size={200}
    />
    <p className="text-sm text-gray-500">
      Request ID: {request.requestId}
    </p>
  </div>
)}
```

The QR code encodes only the Request ID and not the full approval details. This means the code itself contains no sensitive information. The actual approval status is fetched from the database at the time of scanning, ensuring that even if a QR code image is captured and shared, it will only return an Approved status if the original request still holds that status in the system.

5.4.2 QR Code Scanning and Verification

The Security Guard dashboard includes a scanning interface built using the `Html5QrcodeScanner` from the HTML5 QR Code library. When the guard initiates a scan, the library requests access to the device camera through the browser's native media API and renders a live viewfinder in the designated section of the dashboard. Once a QR code is detected within the camera frame, the library decodes it and returns the encoded string — in this case, the Request ID.

```typescript
// QR Scanner initialisation on security dashboard
useEffect(() => {
  const scanner = new Html5QrcodeScanner('reader', {
    fps: 10,
    qrbox: { width: 250, height: 250 },
  }, false);

  scanner.render(
    async (decodedText) => {
      scanner.clear();
      const res = await fetch(
        `/api/requests/verify?requestId=${decodedText}`
      );
      const data = await res.json();
      setScanResult(data);
    },
    (error) => { console.warn(error); }
  );
}, []);
```

The decoded Request ID is sent to the `/api/requests/verify` endpoint, which queries the database for the matching request and returns its current overall status along with the student's name and request details. The guard's dashboard then displays this information clearly, with colour-coded status indicators — green for Approved, red for Denied, and amber for Pending. Each completed scan is recorded in the Scans collection with the guard's user ID and a timestamp, creating a permanent log of all gate exit verifications.

5.5 Database Integration and Deployment

5.5.1 MongoDB Connection Management

Since Next.js API Routes are serverless functions, a new database connection would be initiated on every request if not handled carefully, which would exhaust MongoDB's connection limits under moderate usage. To prevent this, a singleton connection pattern is implemented in `/lib/dbConnect.ts`. This function checks whether a connection is already

established and cached in the global scope before attempting to create a new one, ensuring that the application maintains a single reused connection throughout its active lifecycle.

```typescript
// Singleton database connection
import mongoose from 'mongoose';

let cached = global.mongoose;

if (!cached) {
  cached = global.mongoose = { conn: null, promise: null };
}

export async function dbConnect() {
  if (cached.conn) return cached.conn;
  if (!cached.promise) {
    cached.promise = mongoose.connect(process.env.MONGODB_URI!);
  }
  cached.conn = await cached.promise;
  return cached.conn;
}
```

5.5.2 Mongoose Models

The three collections described in Chapter 4 — Users, Requests, and Scans — are each defined as Mongoose models in the `/models` directory. Mongoose schemas enforce field types, required fields, default values, and enumerated options at the application level before any document is written to the database. For example, the role field in the User schema is defined with an enum constraint that only permits the values `student`, `hod`, `warden`, `security`, and `admin`, preventing invalid role assignments.

5.5.3 Environment Variables

All sensitive configuration values — the MongoDB connection string, the NextAuth secret key, and the application base URL — are stored as environment variables and never committed to the codebase. In the local development environment, these are defined in a `.env.local` file. In the Vercel production environment, they are configured through the Vercel project settings dashboard.

5.5.4 Deployment on Vercel

The application is deployed on Vercel by connecting the project's GitHub repository to a Vercel project. Every push to the main branch triggers an automatic build and deployment pipeline. Vercel builds the Next.js application, runs the TypeScript compiler to check for type errors,

and deploys the output to its global edge network. The deployed application is accessible at *https://gatevault-agps.vercel.app*. Vercel's serverless function infrastructure handles the API routes without requiring any manual server provisioning or configuration, and its automatic scaling ensures that the application can handle concurrent users without performance degradation.

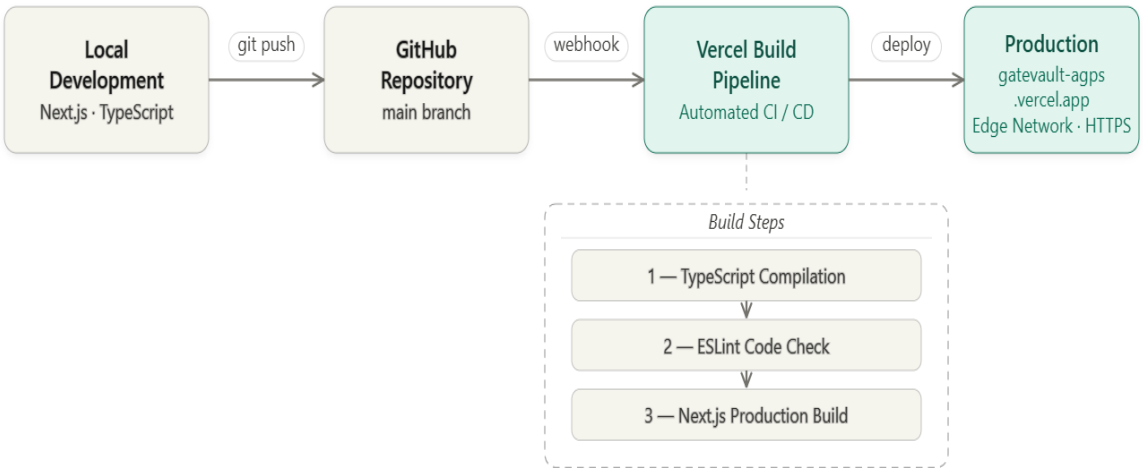

Figure 5.2: Deployment Pipeline — GitHub to Vercel Production

Chapter 6

Results and Discussion

This chapter presents the results of the system implementation through a structured walkthrough of each role-specific module. For every module, the interface is described in relation to the functional requirements defined in Chapter 3, and the extent to which those requirements have been fulfilled is discussed.

6.1 Student Dashboard

The Student module serves as the primary interface through which the end-user — the hostel student — interacts with the system. Upon successful login, the student is directed to their personal dashboard, which is organised into four distinct sections: New Request, Active Requests, QR Code, and History.

6.1.1 Login and Landing

The login page presents a clean, centred form requesting the user's registered email address and password. Authentication is validated against the database, and an incorrect credential entry returns an inline error message without refreshing the page. On successful login, the system reads the authenticated user's role from the session token and redirects them to the appropriate dashboard — a student is never routed to the warden or HoD interface regardless of the URL they attempt to access manually.

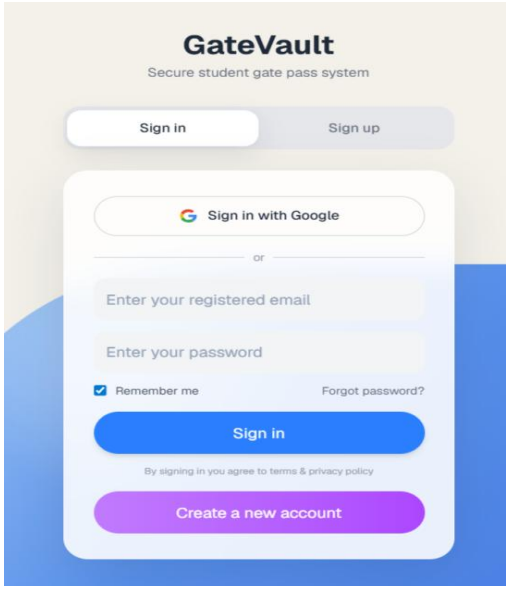

Figure 6.1: Login Page — GateVault

6.1.2 Form Submission

The New Request section allows the student to choose between two request types — Gate Pass and Leave — through a toggle or tab interface. The Gate Pass form collects the reason for exit, destination, and intended return time. The Leave form additionally collects the date range of the absence. Both forms include client-side validation that prevents submission of empty or improperly formatted fields. On submission, the system assigns a unique Request ID to the entry and displays a confirmation to the student. The dashboard then transitions to show the newly created request in the Active Requests section with a status of Pending.

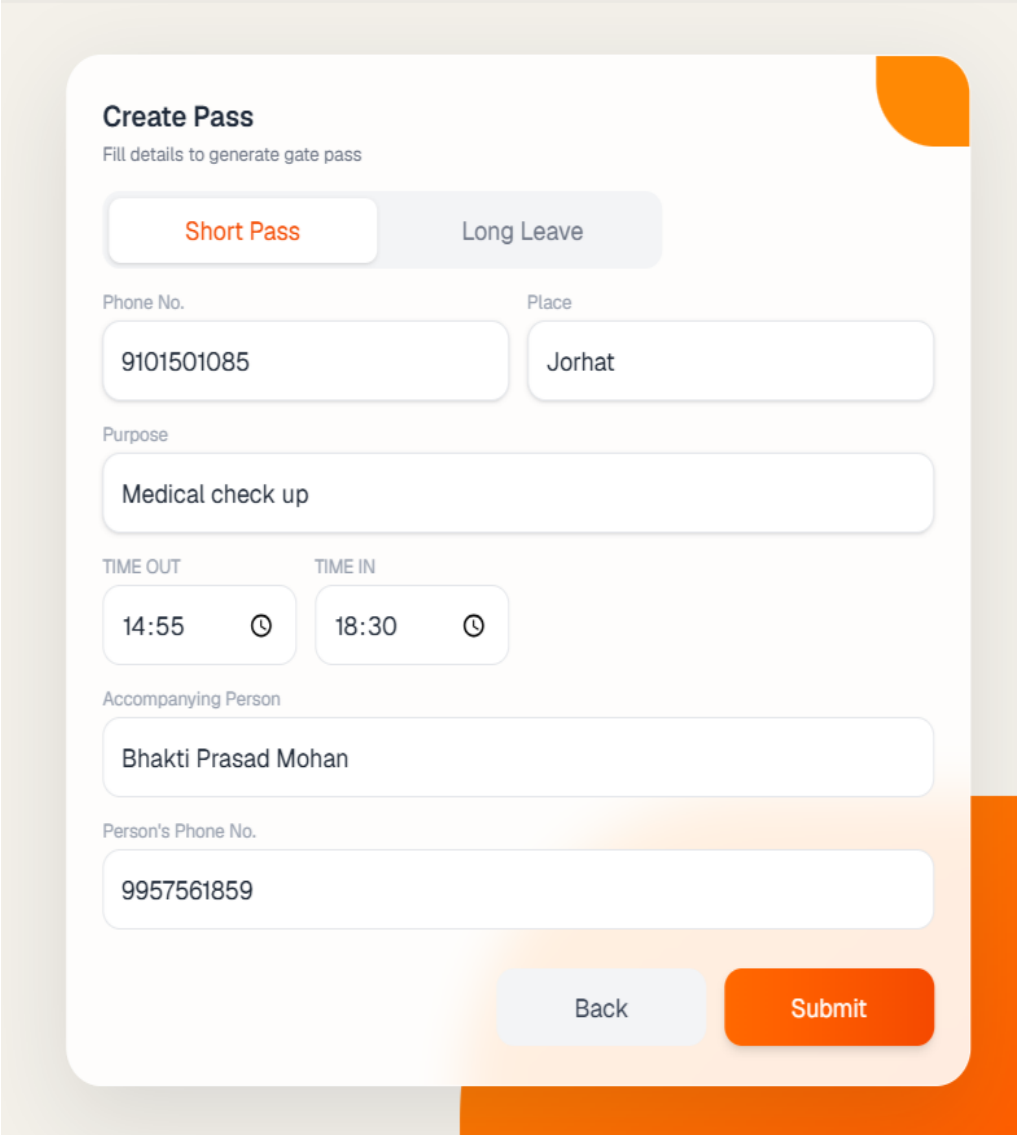

Figure 6.2: Student Dashboard — Gate Pass Request Submission Form

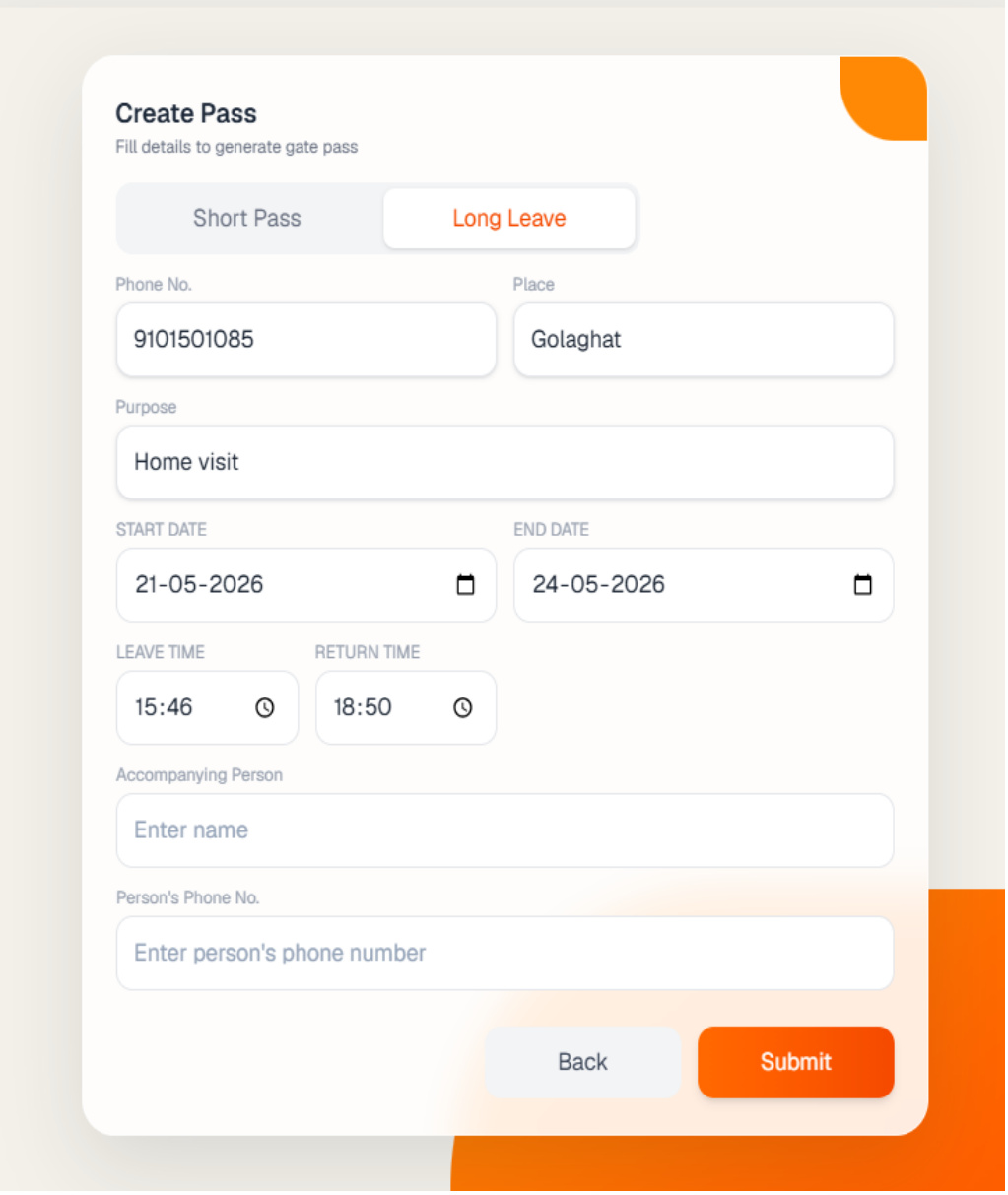

Figure 6.3: Student Dashboard — Leave Request Submission Form

6.1.3 Status Tracking

The Active Requests section displays all requests that are currently in progress — those that have not yet been fully resolved through approval, denial, or completion. Each request card shows the Request ID, the type of request, the submission date, and the current status. For leave requests on working days, where two approvals are required, the card displays both the HoD status and the Warden status independently, allowing the student to see precisely where in the approval chain their request currently sits. This directly addresses the transparency problem identified in the problem statement, where students in the manual system had no visibility into the progress of a submitted form.

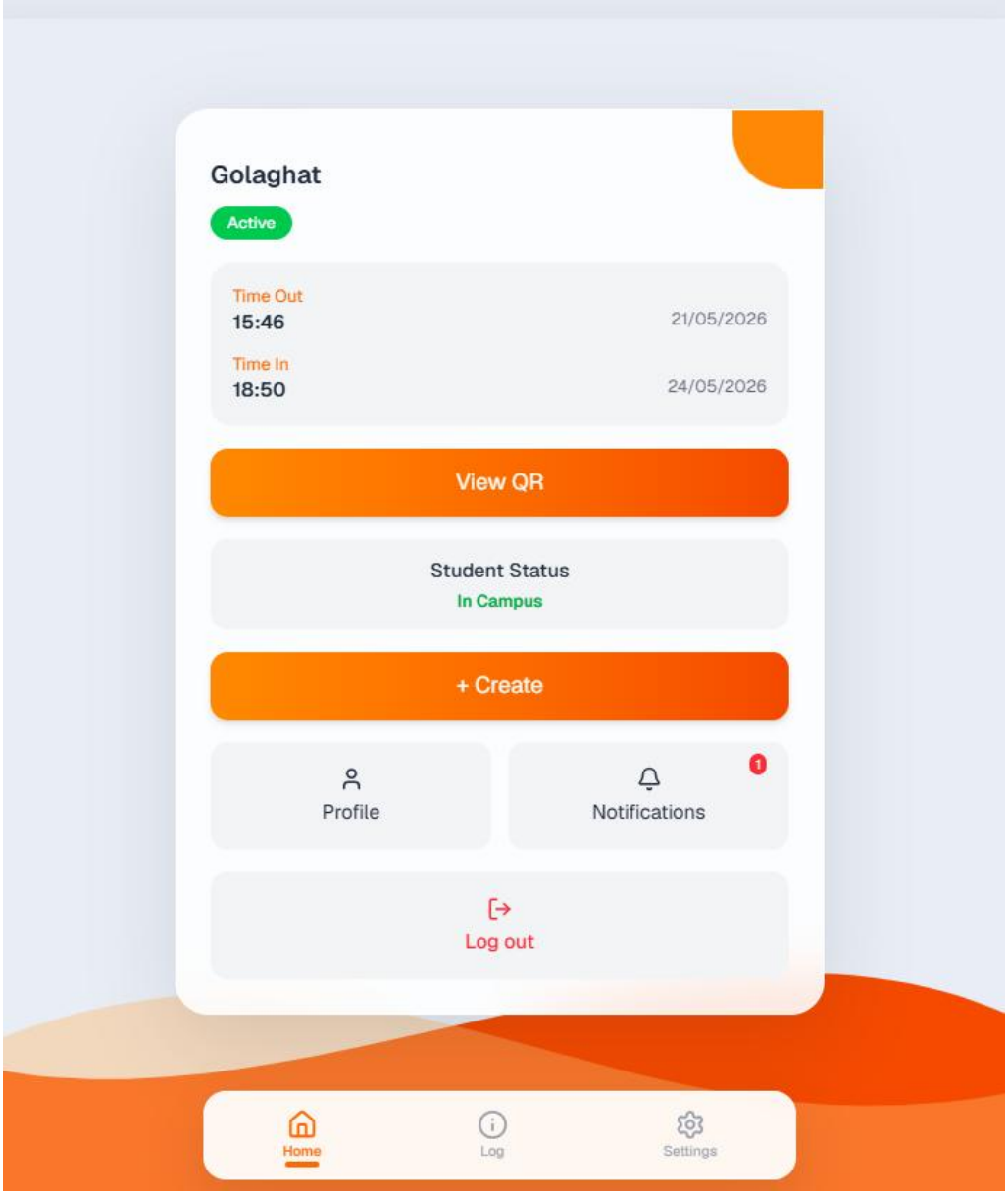

Figure 6.4: Student Dashboard — Active Requests with Status Indicators

6.1.4 QR Code Display

When a request receives full approval, the status card for that request updates to Approved and a View QR Code button becomes visible. Clicking this button opens the QR code generated for that specific request. The code is rendered at a sufficient size for reliable camera scanning and is accompanied by the associated Request ID displayed in text below it. The student presents this screen to the Security Guard at the gate.

The QR code is visible only for requests with an overall approved status. Requests that are pending or denied do not generate a QR code under any circumstances, ensuring that the gate verification step cannot be reached without a legitimate approval.

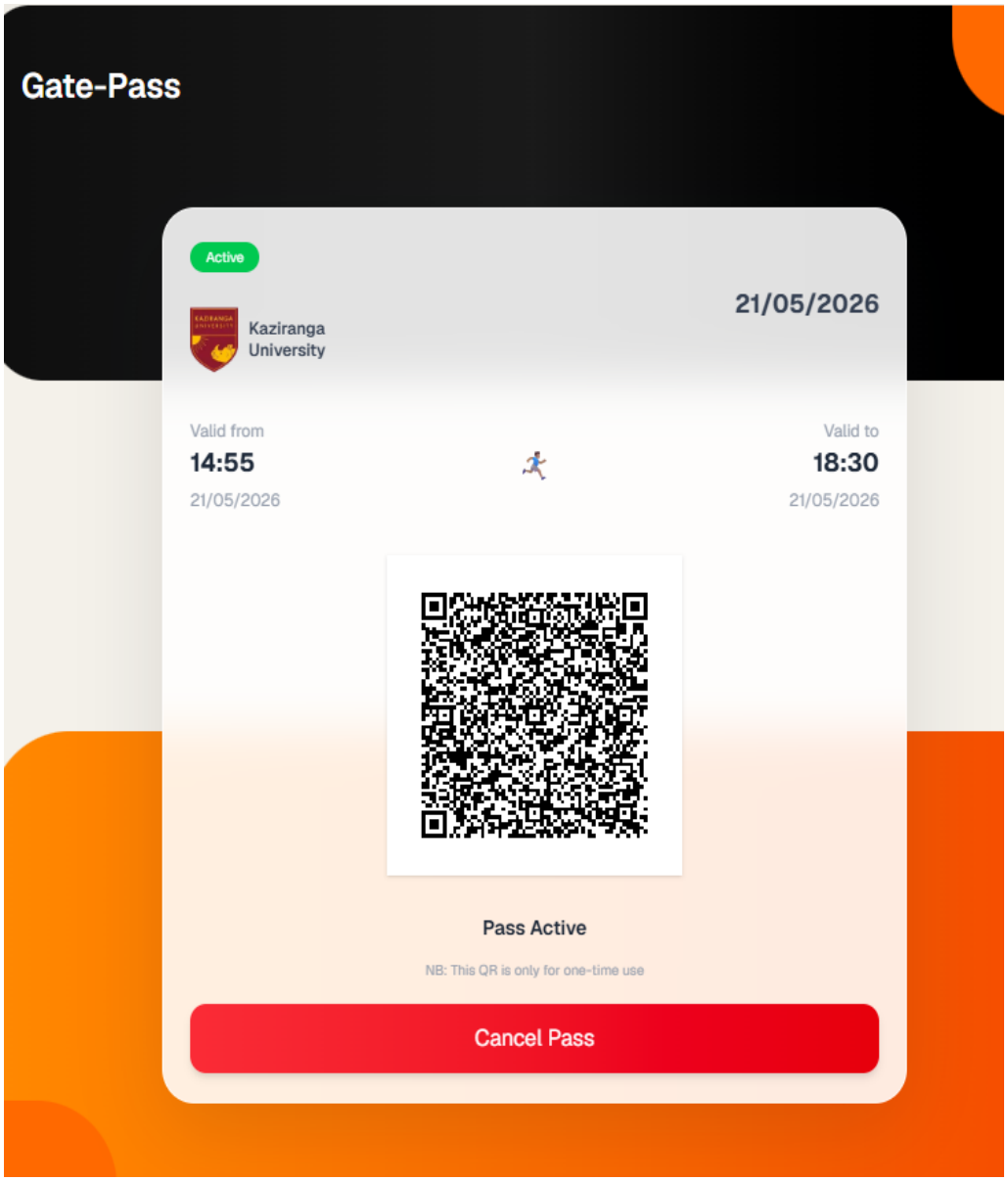

Figure 6.5: Student Dashboard — Approved Request with QR Code Display

6.1.5 Request History

The History section maintains a complete log of all past requests submitted by the student, including those that were denied or have been completed. Each entry shows the request type, submission date, destination, and final status. This gives students a personal record of their movement history, which may also be useful as documentation in cases where an authority queries a student's past requests.

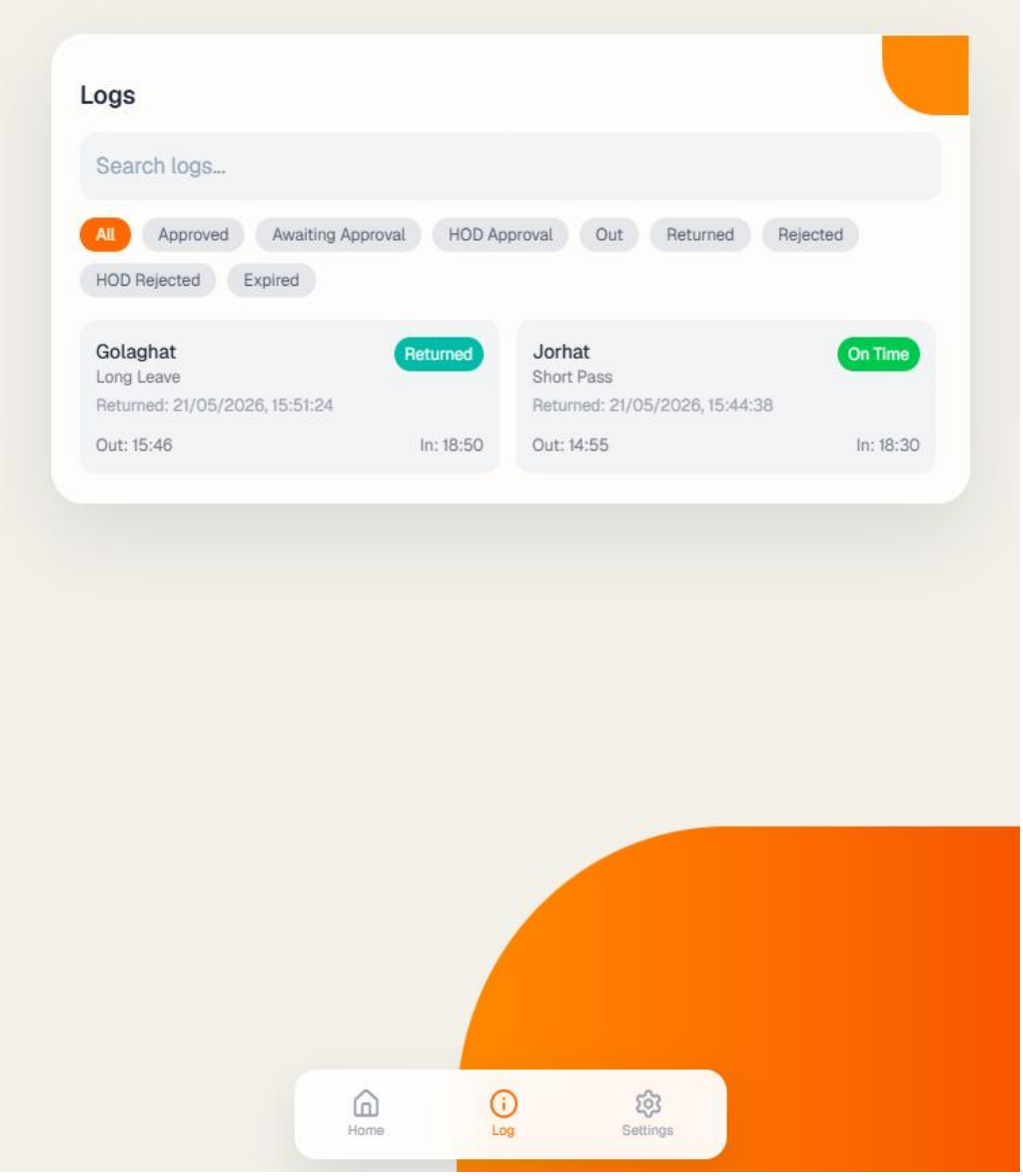

Figure 6.6: Student Dashboard — Request History Log

6.2 HoD and Warden Dashboard

The HoD and Warden modules share a structurally similar interface, as both serve an approval function within the system. The primary difference lies in what each role receives — the HoD receives only leave requests on working days, while the Warden receives gate pass requests directly and leave requests only after HoD approval. This section discusses both dashboards together, noting where their behaviour differs.

6.2.1 Incoming Requests Panel

On login, the HoD and Warden are each presented with a dashboard that opens on the Incoming Requests panel by default. This panel lists all requests currently assigned to that authority and awaiting a decision. Each request entry displays the student's name, roll number, request type,

reason, destination, date range, and submission timestamp. The information is presented compactly enough that an authority can assess a request without needing to open a separate detail view for routine decisions.

For the Warden specifically, incoming leave requests are clearly distinguished from gate pass requests through a type label on each card, since both types appear in the same queue. This distinction is important because the approval context for a leave request — which has already been approved by the HoD — differs from a gate pass request, which has not passed through any prior review.

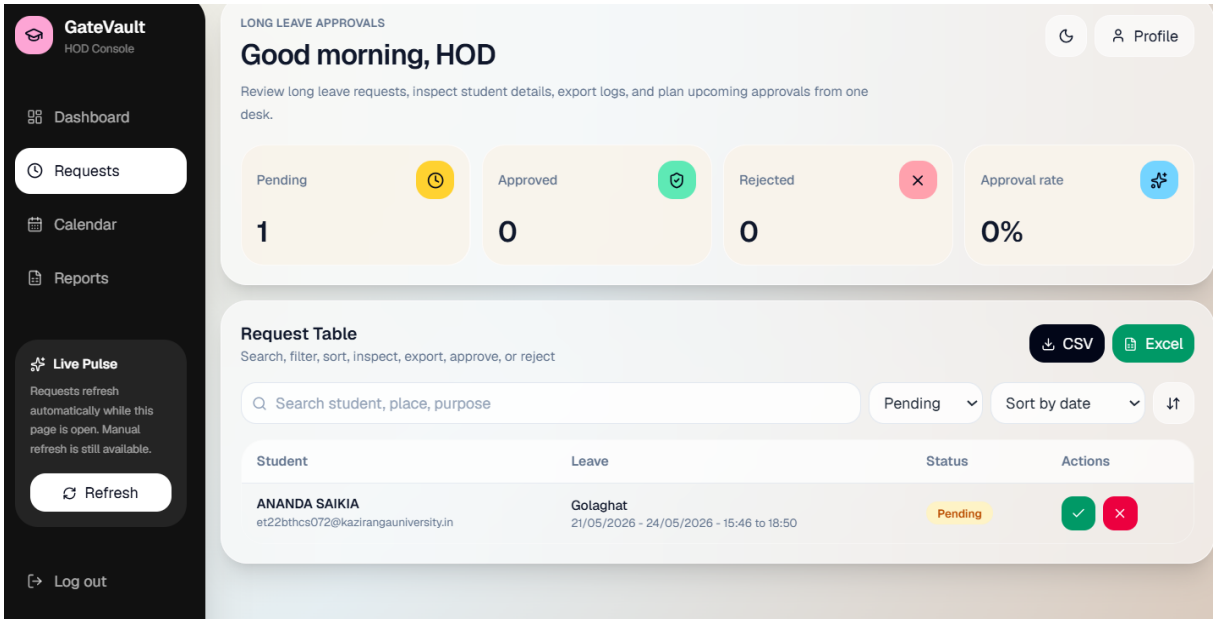

Figure 6.7: HoD Dashboard — Incoming Leave Requests Panel

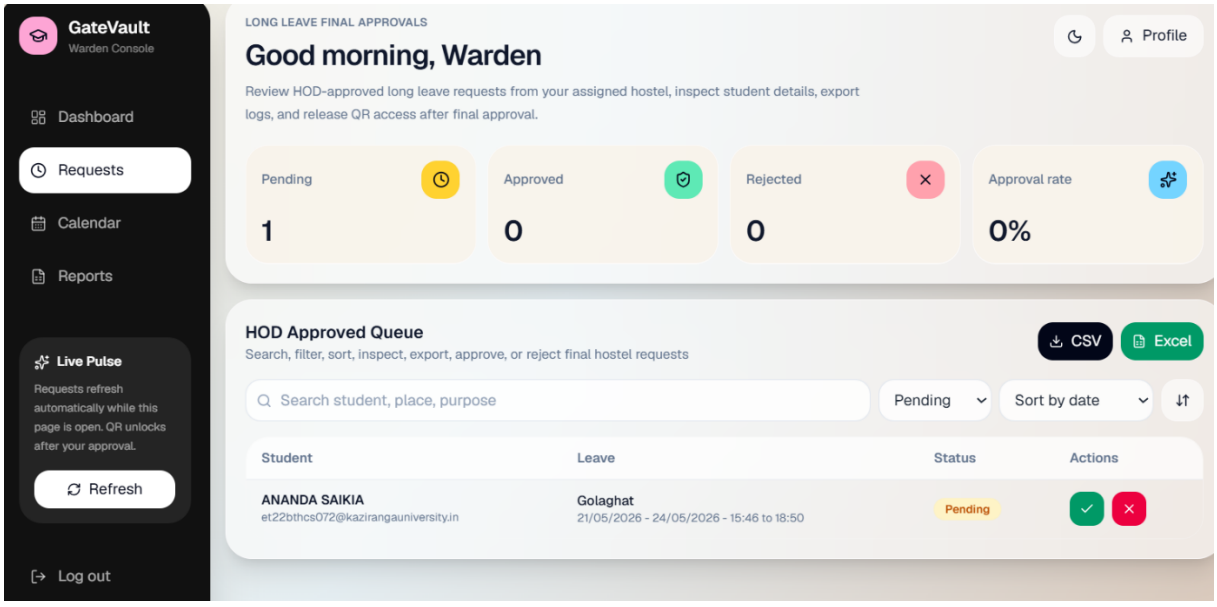

Figure 6.8: Warden Dashboard — Incoming Requests Panel with Mixed Request Types

6.2.2 Approval and Denial Actions

Each request card presents two action buttons — Approve and Deny. On clicking Approve, the system updates the relevant status field in the database. For the HoD, this sets `hodStatus` to approved and automatically forwards the request to the Warden queue. For the Warden, this sets `wardenStatus` to approved and triggers QR code generation for the student. On clicking Deny, the request is closed regardless of its position in the approval chain, and the student's dashboard reflects the denial immediately. A confirmation prompt is shown before a denial is recorded, reducing the risk of accidental rejection.

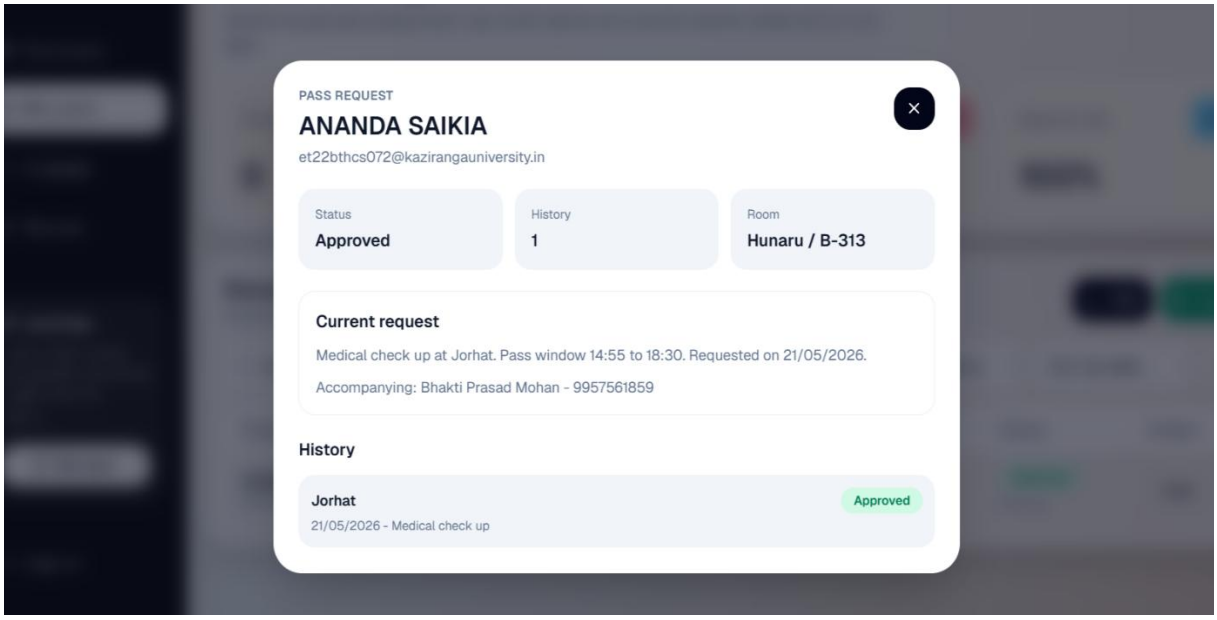

Figure 6.9: Warden Dashboard — Approval Action on a Pending Request

6.2.3 Search Functionality

Both dashboards include a search bar that filters the request list by student name. This is particularly useful during high-volume periods — such as semester breaks — when a large number of requests accumulate simultaneously. Rather than scrolling through the full list, the authority can type a student's name and immediately see only that student's pending requests. The search operates on the client side for already-loaded data, making it instantaneous.

6.2.4 History and Light/Dark Mode

A History tab on both dashboards shows all past decisions made by that authority, with timestamps. This provides an audit trail that administrators can cross-reference if a dispute arises about whether a request was approved or denied. Both dashboards also support a light and dark mode toggle, which persists in the user's browser session and adjusts the entire interface colour scheme accordingly.

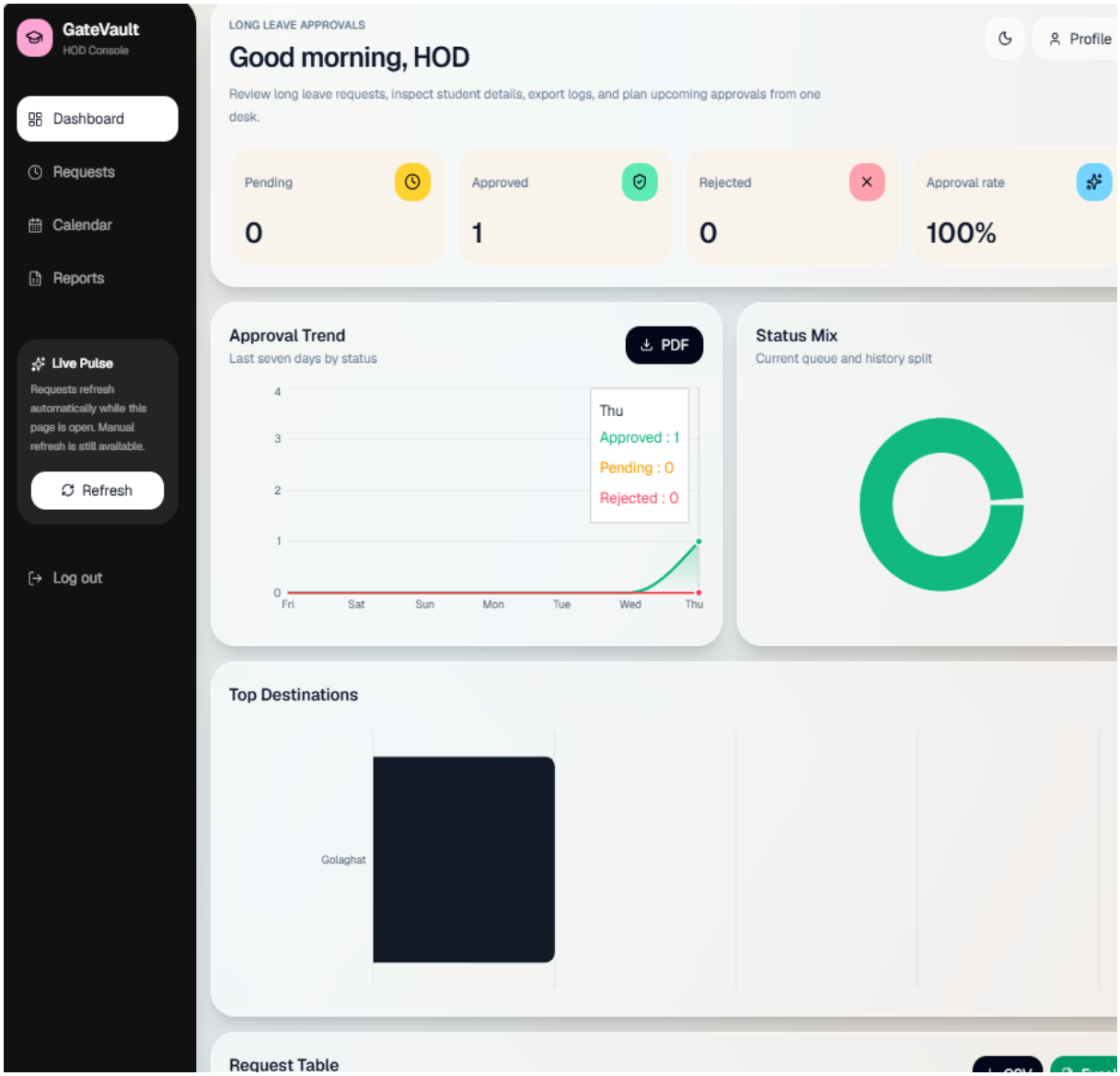

Figure 6.10: HoD Dashboard — Approval History Log

6.3 Security Guard Module

The Security Guard module is purpose-built for a single primary function — scanning a student's QR code at the campus gate and returning a clear, immediate verification result. The interface is intentionally minimal, as the guard needs to perform this action quickly and under conditions that may not always be ideal, such as outdoors in varying light or during a busy exit period.

6.3.1 QR Scanning Interface

On login, the Security Guard is directed to their dashboard, which presents the QR scanning interface prominently at the top of the page. A Scan QR Code button activates the device camera through the browser's media API and renders a live viewfinder with a defined scanning

frame. The student holds their QR code up to the camera, and the system detects and decodes it within one to two seconds under normal lighting conditions.

Once decoded, the Request ID is sent to the verification API, which queries the database and returns the request details along with the current overall status. The result is displayed immediately below the scanner with a colour-coded status indicator — a green badge for Approved, a red badge for Denied, and an amber badge for Pending. The student's name, request type, and destination are displayed alongside the status, giving the guard enough context to make an informed decision at the gate.

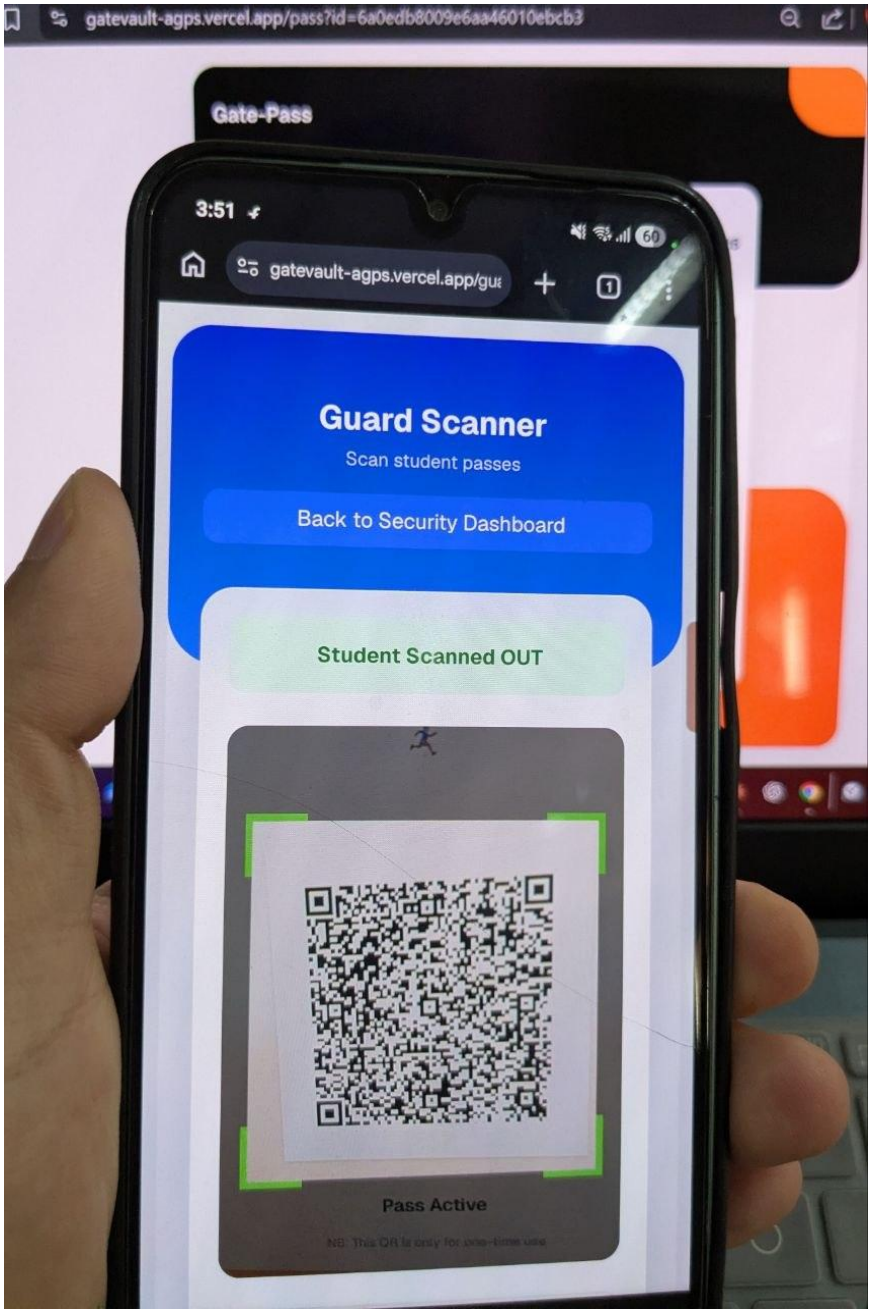

Figure 6.11: Security Guard Dashboard — QR Scanner Active with Viewfinder

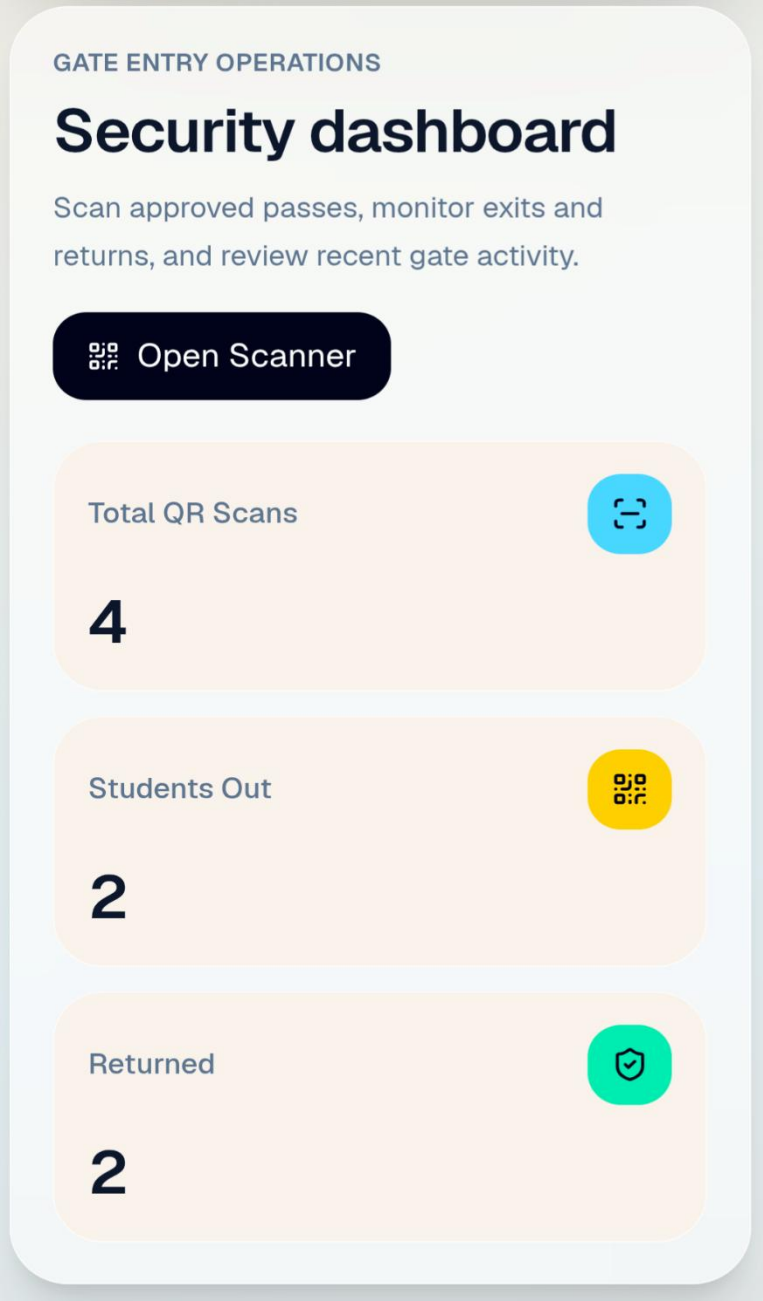

Figure 6.12: Security Guard Dashboard — Scan Result Displaying Approved Status

6.3.2 Scan History

Below the scanner, the dashboard maintains a session-based log of all QR codes scanned since the guard logged in. Each entry shows the student's name, the request type, the result, and the time of the scan. This log allows the guard to refer back to a previous scan if a student returns to the gate with a query about an earlier verification.

The total number of scans completed in the current session is displayed at the top of the history section as a running count, which gives the guard a quick reference for the volume of exits processed during their shift.

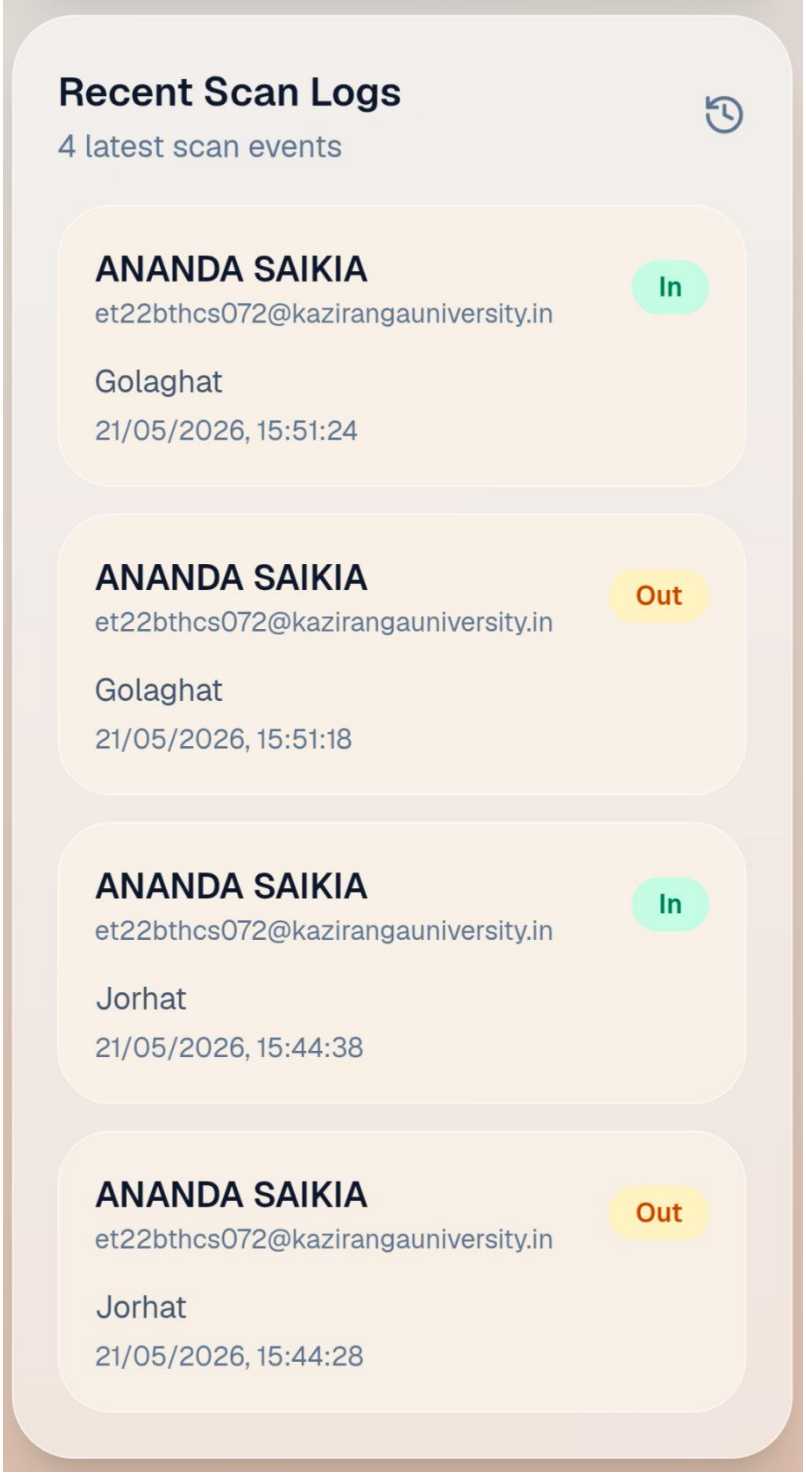

Figure 6.13: Security Guard Dashboard — Scan History Log

6.4 Admin Dashboard

The Administrator dashboard provides institution-level oversight of the entire system. The Admin does not participate in the approval workflow but has read access to all data across all students, requests, and user accounts. This module is intended for use by an institutional administrator or the head warden responsible for the hostel management system as a whole.

6.4.1 System Overview

The Admin dashboard opens with a summary panel that displays aggregate data — the total number of requests submitted to date, the count broken down by status (Pending, Approved, Denied), and the number of registered users by role. A bar chart built using Recharts visualises the request status distribution, giving the administrator an at-a-glance picture of system activity without requiring them to scroll through individual records.

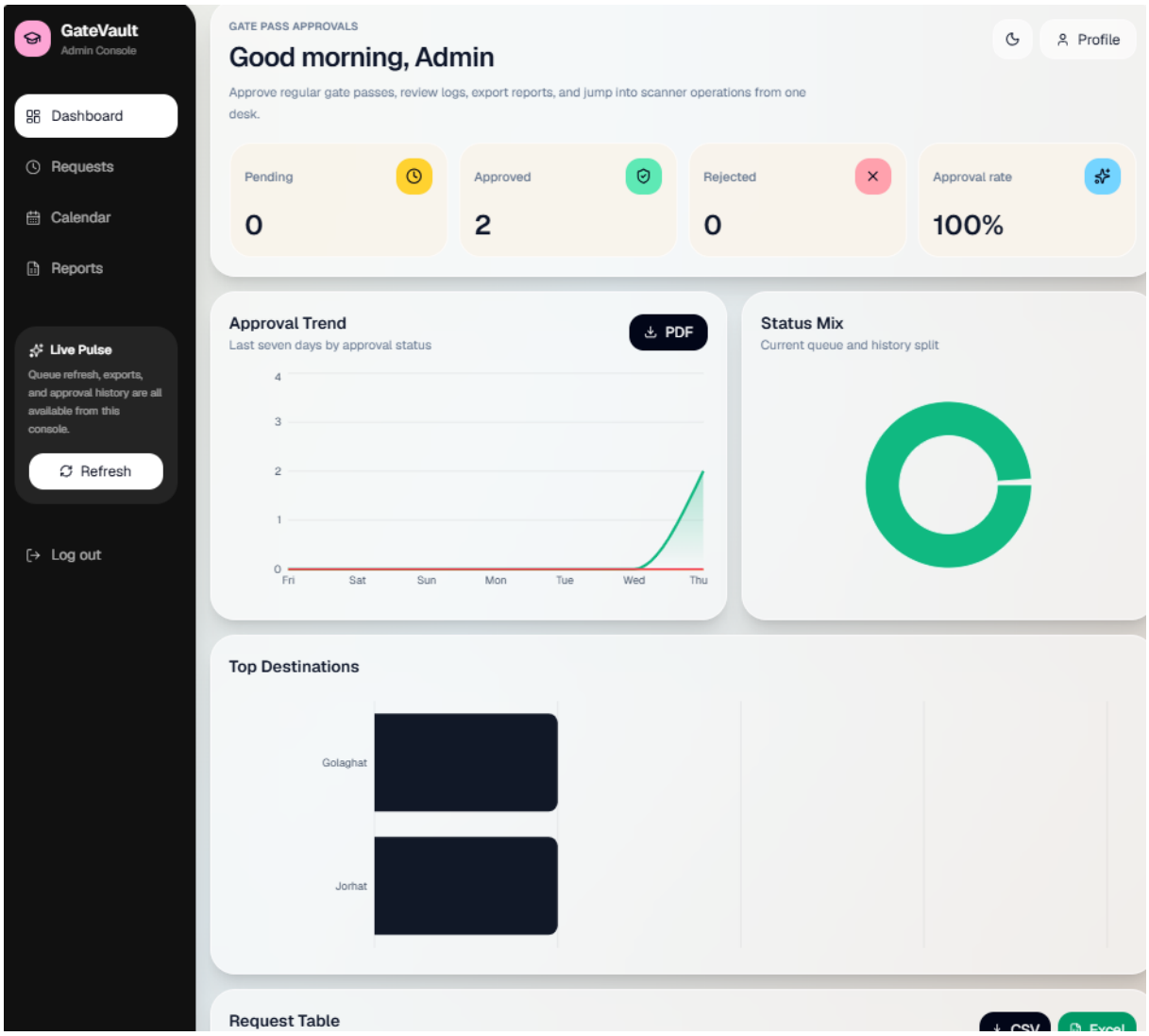

Figure 6.14: Admin Dashboard — System Overview with Summary Statistics and Chart

6.4.2 Student Search and Full Records

The Admin can search for any student by name and view all requests associated with that student, including the full approval chain, timestamps, and the identity of the approving authorities at each stage. This capability is particularly useful in cases where an institution needs to verify a student's movement history for disciplinary or welfare purposes. The search results display all requests chronologically, with the most recent at the top.

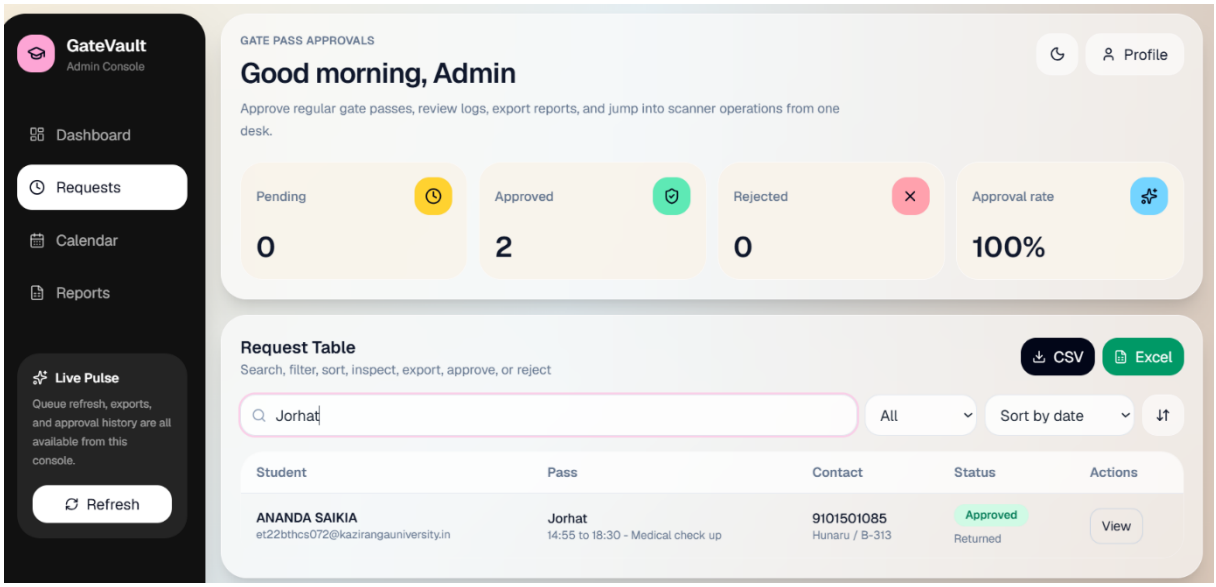

Figure 6.15: Admin Dashboard — Student Search Results with Full Request Details

6.4.3 Status List and History

A Status List view shows all currently active requests across all students, filtered by status. The administrator can toggle between views to see only Pending requests, only Approved requests, or all requests together. A separate History view shows the complete record of all closed requests — those that were denied, completed, or cancelled — along with the dates and actions associated with each.

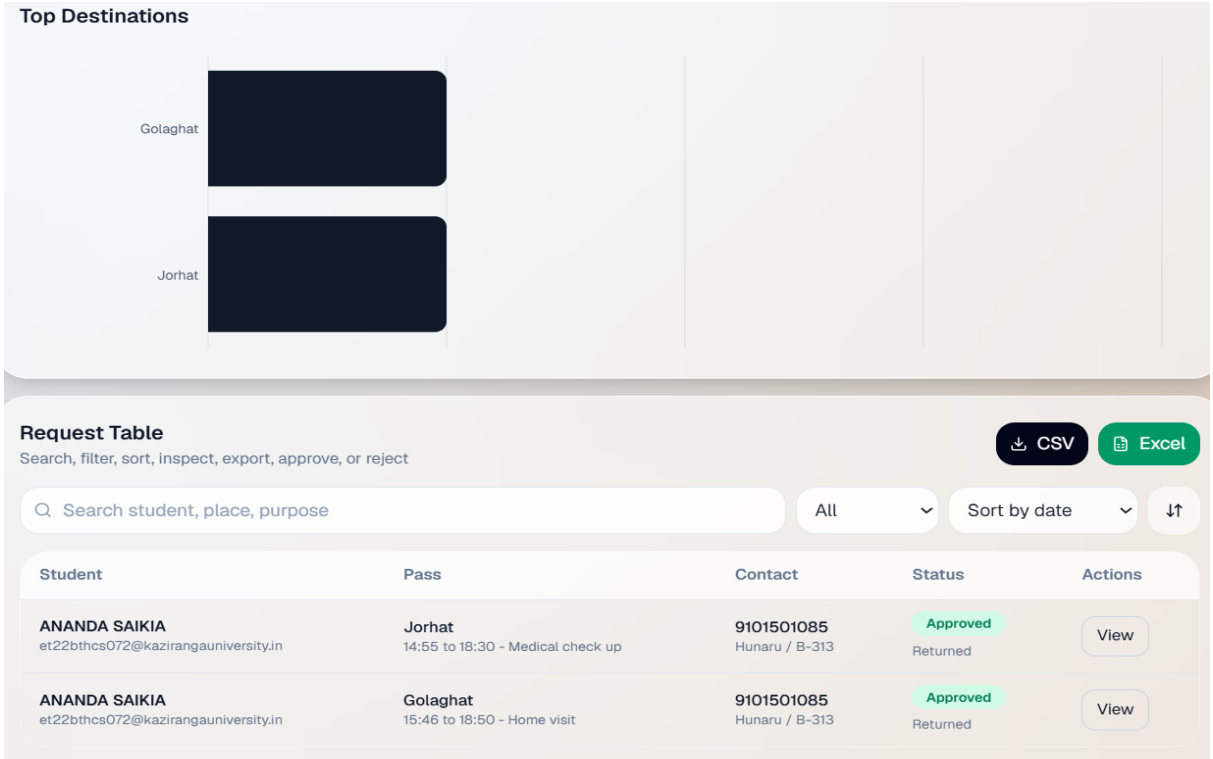

Figure 6.16: Admin Dashboard — Full System Request History

Discussion

The results presented in this chapter demonstrate that all five user modules function as specified in the functional requirements defined in Chapter 3. The end-to-end workflow — from student form submission through role-based approval to QR-based gate verification — operates without manual intervention at any stage, fulfilling the primary objective of building a paperless and automated gate pass management system.

The role separation enforced through NextAuth.js sessions and server-side middleware ensures that each user interacts only with the data and actions relevant to their position in the workflow. No cross-role data access was observed during testing, confirming that the RBAC implementation is functioning as intended.

The QR code verification module introduces a meaningful security improvement over the manual system. Because the QR code encodes only the Request ID and the actual approval status is fetched live from the database at the time of scanning, a student cannot present an outdated or fabricated code to gain exit approval. The guard's interface returns a clear, real-time result, which reduces the scope for human error or judgement calls at the gate.

The mobile-responsive design was validated across multiple device sizes. All dashboards, forms, and the QR scanning interface functioned correctly on both Android and iOS devices tested through Chrome and Safari, consistent with the non-functional responsiveness requirement.

One limitation observed during results evaluation is that the QR scanning interface requires a stable internet connection at the point of verification. In instances of network disruption, the scan can decode the QR code but the API call to verify the status will fail. An offline fallback mechanism — such as temporarily caching recently approved Request IDs on the device — has been identified as a priority for future development and is discussed in Chapter 8.

Chapter 7

Testing

Testing is a critical phase in the software development lifecycle that verifies whether the implemented system behaves as intended and meets the requirements established during the analysis phase. For GateVault, testing was conducted across three levels — unit testing, integration testing, and system testing — to validate individual components, their interactions, and the end-to-end behaviour of the application as a whole. This chapter describes the testing approach adopted and presents the test cases executed along with their outcomes.

7.1 Test Cases and Results

7.1.1 Unit Testing

Unit testing was carried out on individual functions and components in isolation, without dependency on other parts of the system. The primary focus at this level was on the backend utility functions that carry business-critical logic — specifically, the approval path determination function, the working day check, the password hashing and comparison operations using bcryptjs, and the Mongoose schema validation rules. Each function was tested with both valid inputs and boundary or invalid inputs to verify that it returned the expected output in all cases. No unit testing framework was formally integrated into the project; tests were executed manually and through console verification during development.

7.1.2 Integration Testing

Integration testing was performed to verify that the frontend, backend API routes, and database worked together correctly as connected layers. Each API endpoint was tested using both correct and malformed request payloads to confirm that the server responded appropriately in every case — returning the expected data on success and a meaningful error response on failure. Particular attention was given to the request routing logic, the session and role validation middleware, and the sequence of database writes triggered by an approval action.

7.1.3 System Testing

System testing was conducted on the fully deployed application at *https://gatevault-agps.vercel.app* to validate the complete end-to-end workflows under real conditions. Each user role was tested independently using a dedicated test account, and the full lifecycle of both

request types — Gate Pass and Leave — was executed from submission through to gate verification. The mobile responsiveness of each dashboard was also verified on physical devices during this phase.

The test cases executed across all three levels are documented in the tables below, organised by module.

Table 7.1: Authentication Module — Test Cases and Results

TC ID	Test Description	Input / Action	Expected Result	Actual Result	Status
TC-01	Login with valid credentials	Registered email address and correct password submitted via login form	User is authenticated and redirected to their role-specific dashboard	Redirected correctly to role-appropriate dashboard on every attempt	Pass
TC-02	Login with incorrect password	Registered email with a wrong password submitted	Inline error message displayed; no session created and no redirect occurs	Error message shown on the login page; page remained static	Pass
TC-03	Login with unregistered email	An email address not present in the Users collection submitted	Error message displayed; authentication rejected	Appropriate error message returned; no session initiated	Pass
TC-04	Role-based dashboard redirect on login	Valid student credentials used to log in to the system	User is directed exclusively to the Student dashboard	Student dashboard loaded; no access to any other role's dashboard	Pass
TC-05	Manual URL access to another role's dashboard	Student session active; Warden dashboard URL entered manually in the browser	Access denied; user redirected back to Student dashboard	Server-side role check triggered; redirect to Student dashboard executed	Pass
TC-06	Session expiry behaviour	Authenticated user remains idle beyond the session timeout window	User redirected to login page on next action attempt	Expired session detected by NextAuth; login page loaded on next navigation	Pass

Table 7.2: Student Module — Test Cases and Results

TC ID	Test Description	Input / Action	Expected Result	Actual Result	Status
TC-07	Gate Pass submission with all valid fields	Complete Gate Pass form with reason, destination, and return time submitted	Request created in database, unique RID assigned, and status shown as Pending	Request created with correct RID; Pending status displayed on dashboard	Pass
TC-08	Leave form submission with valid date range	Complete Leave form with departure and return dates submitted on a working day	Request created and automatically routed to HoD queue for first-stage approval	Request routed correctly to HoD dashboard; HoD status shown as Pending	Pass
TC-09	Form submission with missing required fields	Gate Pass form submitted with the destination field left empty	Inline validation error displayed; form submission blocked; no database write	Error highlighted on the empty field; page remained on form without submitting	Pass
TC-10	Real-time status update after HoD approval	HoD approves a Leave request in a separate browser session	Student dashboard status updates to reflect HoD Approved stage	Status updated to HoD Approved upon page refresh on the student dashboard	Pass
TC-11	QR code not visible before approval	Pending request viewed on the Student dashboard	View QR Code button is not rendered on the request card	QR Code button absent from the pending request card as expected	Pass
TC-12	QR code generation after full approval	Warden approves a Gate Pass request; student views their dashboard	QR code visible and scannable on the approved request card	QR code generated and displayed correctly; successfully scanned in verification test	Pass
TC-13	History log completeness	Student account with multiple past requests of varying outcomes views History tab	All past requests listed chronologically with correct final status for each	All records displayed with accurate status labels and submission timestamps	Pass

Table 7.3: HoD and Warden Module — Test Cases and Results

TC ID	Test Description	Input / Action	Expected Result	Actual Result	Status
HOD MODULE					
TC-14	Leave request appears in HoD queue on a working day	Student submits a Leave request on a weekday; HoD logs in to their dashboard	Submitted Leave request is visible in the HoD incoming requests panel	Request appeared in the HoD queue with correct student name, reason, and date range	Pass
TC-15	Gate Pass request does not appear in HoD queue	Student submits a Gate Pass request; HoD dashboard inspected for the submission	Gate Pass request is absent from the HoD incoming panel entirely	Request correctly excluded from HoD queue; routed directly to Warden queue instead	Pass
TC-16	HoD approval automatically forwards request to Warden	HoD clicks Approve on a pending Leave request in their dashboard	Request moves to Warden queue; HoD status field locked at Approved in the database	Request forwarded to Warden queue immediately; hodStatus set to approved and immutable	Pass
TC-17	HoD denial closes the request at first stage	HoD clicks Deny on a pending Leave request after reviewing the submission	Request removed from HoD active queue; student dashboard reflects Denied status; request does not reach Warden	Request closed immediately; student dashboard updated to Denied; Warden queue unaffected	Pass
WARDEN MODULE					
TC-18	Warden approval triggers QR code generation	Warden clicks Approve on a pending Gate Pass request in their dashboard	System generates a QR code and makes it available on the student's dashboard	QR code generated correctly and displayed on the student dashboard; successfully scanned in a subsequent test	Pass
TC-19	Warden cannot act on a Leave request without prior HoD approval	Leave request submitted on a working day; Warden dashboard inspected before HoD acts	Leave request is absent from Warden queue until HoD approval is recorded	Request not visible in Warden queue; appeared only after HoD approval was given in a follow-up action	Pass

TC-20	Search by student name filters the request list	Warden types a specific student name into the search bar on their dashboard	Request list filtered to display only requests submitted by the named student	Filtered results returned correctly; requests from other students excluded from the view	Pass
TC-21	Approval history log accuracy	Warden navigates to the History tab after processing several requests	All past approvals and denials listed in chronological order with accurate timestamps and student names	Complete history log displayed with correct decision outcomes and precise timestamps for each entry	Pass

Note — TC-14 through TC-17 were executed using a dedicated HoD test account. TC-18 through TC-21 were executed using a dedicated Warden test account on the live Vercel deployment.

Table 7.4: Security Guard Module — Test Cases and Results

TC ID	Test Description	Input / Action	Expected Result	Actual Result	Status
QR CODE VERIFICATION OUTCOMES					
TC-22	QR scan of a fully approved request	Guard scans the QR code of a request with overallStatus = approved	Approved status badge displayed alongside the student name, request type, and destination	Green Approved badge rendered with full request details; result returned within one second	Pass
TC-23	QR scan of a denied request	Guard scans the QR code of a request with overallStatus = denied	Denied status badge displayed with student details; exit should not be permitted by the guard	Red Denied badge displayed correctly with student name and request details visible	Pass
TC-24	QR scan of a still-pending request	Guard scans the QR code of a request with overallStatus = pending	Pending status badge displayed; guard is informed the approval chain is incomplete	Amber Pending badge rendered correctly; guard dashboard indicated that approval was not yet complete	Pass
TC-25	QR scan of an invalid or unrecognised code	Guard scans a QR code that does not correspond to any request document in the database	An error message is displayed indicating that the request could not be found; no access permitted	Error response returned from the verification API; dashboard displayed a clear request-not-found message	Pass

INTERFACE AND SESSION HISTORY					
TC-26	Scan history log updated after each successive scan	Guard performs three successive QR scans on different request codes during the same session	All three scan events appear in the session history log with student names, outcomes, and timestamps	All three entries recorded in the scan history log in correct chronological order after each scan	Pass
TC-27	Camera access permission prompt on scan initiation	Guard taps the Scan QR Code button on the Security Guard dashboard for the first time	Browser displays a native camera permission request before activating the live viewfinder	Camera permission prompt appeared correctly on both Android Chrome and iOS Safari during testing	Pass

Note — TC-27 was verified on both Android (Chrome) and iOS (Safari) to confirm cross-platform camera API compatibility. All verification results were returned from the live MongoDB Atlas database in under two seconds.

Table 7. 5: Admin Module and System-Level Tests

TC ID	Test Description	Input / Action	Expected Result	Actual Result	Status
ADMIN MODULE					
TC-28	Admin full system view on login	Admin account used to log in; system overview dashboard loaded	All requests from all students across all roles visible in the Admin dashboard without filtering	All request records displayed correctly; summary statistics and status chart rendered accurately	Pass
TC-29	Admin search by student name	Admin enters a specific student name in the search field of the Admin dashboard	All requests associated with the named student displayed; records from other students excluded	Filtered results returned correctly with full request details, approval stages, and timestamps for the searched student	Pass
TC-30	Admin cannot perform approval actions	Admin opens a pending request to inspect its details	Approve and Deny action buttons are absent from the request view; Admin has read-only access	No action buttons rendered on the Admin request view; request details displayed in read-only format as expected	Pass

END-TO-END LIFECYCLE TESTS					
TC-31	Complete Gate Pass lifecycle	Student submits Gate Pass → Warden approves → Student views QR code → Security Guard scans QR code	Full single-stage approval workflow executes without error; Approved status returned on scan	All four stages completed successfully on the live deployment; scan returned Approved status with correct student details	Pass
TC-32	Complete Leave Request lifecycle on a working day	Student submits Leave → HoD approves → Warden approves → Student views QR code → Security Guard scans QR code	Full two-stage approval workflow executes correctly; QR code generated only after Warden approval; Approved status returned on scan	All five stages completed successfully; QR code not generated until both approvals were recorded; scan returned correct Approved status	Pass
RESPONSIVENESS AND CONCURRENCY					
TC-33	Mobile responsiveness — Student dashboard	Student dashboard opened on an Android mobile browser at a 360 px viewport width	All dashboard elements render correctly; form fields, status cards, and QR code display fully functional on mobile	Layout adapted correctly to the mobile viewport; all interactive elements remained accessible and usable	Pass
TC-34	Mobile responsiveness — QR scan interface	Security Guard QR scanner interface opened on a mobile browser; scan attempted on a live QR code	Camera activates via the browser media API; QR code detected and scan result returned correctly on mobile	Camera permission prompt appeared; scanner initialised correctly; QR code detected and verification result returned within two seconds	Pass
TC-35	Concurrent request submissions by two students	Two student accounts submit separate Gate Pass requests simultaneously in different browser sessions	Both requests created as independent documents, each assigned a unique RID, with no data conflict or overwrite	Both requests created successfully with distinct RIDs; each request appeared correctly on its respective student's dashboard	Pass

Overall Testing Summary — All Modules					
Total Test Cases	**Passed**	**Failed**	**Pass Rate**	**Test Environment**	**Testing Period**
35	35	0	100%	Live — *gatevault-agps.vercel.app*	April – May 2026

Note — TC-31 and TC-32 represent the most critical system-level tests as they validate the complete end-to-end workflow under real deployment conditions. Both were executed using live test accounts across all five user roles on the Vercel-hosted application.

Summary of Testing Results

All thirty-five test cases executed across the five modules returned a Pass result. No critical defects were identified during unit, integration, or system testing. The complete Gate Pass and Leave request lifecycles — test cases TC-31 and TC-32 respectively — executed without error from submission through to gate verification, confirming that the end-to-end workflow functions correctly under real deployment conditions.

One observation noted during system testing was that the status update on the student dashboard requires a manual page refresh to reflect the latest approval state, rather than updating in real time through a live connection. This is a known limitation of the current polling-based implementation and has been identified as a candidate for improvement through WebSocket integration in a future version of the system, as noted in Chapter 8.

Overall, the testing phase confirmed that the system meets all functional requirements specified in Chapter 3 and performs reliably across device types and user roles.

Chapter 8

Conclusion and Future Scope

8.1 Conclusion

This project set out to address a practical and persistent problem in residential educational institutions — the inefficiency, opacity, and security vulnerabilities of paper-based gate pass and leave management systems. The objectives defined at the outset were clear: to build a fully digital, role-aware, and verifiable system that could handle the complete lifecycle of a gate pass or leave request without any dependence on physical forms or manual signature chains.

GateVault, the Advanced Gate Pass System developed over the course of this project, meets each of those objectives. The system provides a structured, five-role workflow in which a student's request is automatically routed to the appropriate authority based on the type of request and the day of submission. Approval decisions made by the HoD and Warden are reflected on the student's dashboard in real time, replacing the uncertainty that students in the manual system faced once a form had been handed in. Upon full approval, a dynamic QR code tied to the specific request is generated and presented by the student at the gate, where the Security Guard verifies it against a live database query — eliminating the possibility of forged or duplicated passes.

The technology stack chosen for the project — Next.js, React, TypeScript, MongoDB, and NextAuth.js — proved well-suited to the requirements of a multi-role web application. The unified Next.js framework allowed the frontend and backend to be developed and deployed as a single project, simplifying the architecture and reducing the operational overhead of managing separate services. Deployment on Vercel provided a stable, scalable production environment without requiring any server provisioning on the institution's part.

Testing across thirty-five test cases covering all five modules confirmed that the system behaves correctly under both normal and edge-case conditions. The complete Gate Pass and Leave request lifecycles executed without error on the live deployment, and the application performed correctly across mobile and desktop devices, consistent with the responsiveness requirement.

Taken together, the system demonstrates that the administrative workflow of campus exit management — which has remained largely manual in most Indian educational institutions — can be effectively digitised using modern web technologies within the constraints of an undergraduate project. The result is a more transparent, more secure, and considerably more efficient process for students, faculty, and institution staff alike.

8.2 Future Scope

While GateVault fulfils its defined objectives in its current form, several areas have been identified during the course of development and testing where the system can be meaningfully extended.

Real-Time Status Updates — The current implementation requires the student to refresh their dashboard to see the latest approval status. Integrating WebSockets through a library such as Socket.IO or using Next.js server-sent events would allow the dashboard to update automatically the moment an authority takes action on a request, without any manual intervention from the student.

Push Notifications — Email or SMS notifications sent at key points in the workflow — when a request is submitted, approved, denied, or scanned at the gate — would improve the experience for both students and approving authorities. Integration with services such as Twilio for SMS or Nodemailer for email is a straightforward addition given the existing backend structure. WhatsApp-based notifications through the WhatsApp Business API are also a practical option given their near-universal adoption among students in India.

Offline QR Verification Fallback — As noted in the discussion section of Chapter 6, the QR scanning module requires a live internet connection to return a verification result. Implementing a mechanism that caches recently approved Request IDs on the security guard's device would allow basic verification to continue during network disruptions, with the cache synchronising to the database once connectivity is restored.

Multi-Institution Support — The current system is designed for a single institution. Extending the architecture to support multiple institutions under separate tenant namespaces — with institution-level administrators managing their own users and configurations — would allow the platform to be offered as a shared service across several campuses without requiring separate deployments for each.

Native Mobile Application — Although the web application is fully mobile-responsive, a native application built using React Native would provide a smoother experience on smartphones, particularly for the QR scanning functionality. Native camera access is generally faster and more reliable than browser-based camera access, which would improve the experience for security guards during high-traffic periods at the gate.

Analytics and Reporting for Administrators — The current Admin dashboard provides a summary view of request data but does not offer exportable reports or trend analysis. Adding the ability to export request history as a PDF or Excel file, and to view monthly or semester-level movement trends through an expanded analytics dashboard, would make the system more useful as an institutional record-keeping tool.

Integration with Institution ERP — Many universities operate an Enterprise Resource Planning system that manages student data, attendance, and academic records. Integrating GateVault with an institution's ERP would eliminate the need to maintain a separate user database, ensure that student records remain consistent across systems, and allow gate pass data to be considered alongside attendance records when evaluating a student's academic standing.

Parent Notification System — For institutions that maintain parent communication channels, an automated notification to a student's registered parent or guardian at the time a gate pass or leave request is approved would add a layer of safety oversight. This feature would be particularly relevant for junior undergraduate students in their first year of hostel residence.

These extensions represent a natural progression from the current system and are grounded in the practical limitations and operational requirements observed during the development and deployment of GateVault. The existing architecture, built on a modular and well-documented technology stack, provides a solid foundation on which each of these additions can be implemented incrementally.

References

The following bibliography lists all sources cited across the report, numbered in the order of their first appearance in the text. References follow the APA citation format as required by the department guidelines.

[1] Patel, R., Mehta, S., & Joshi, D. (2019). Online hostel gate pass management system. *International Journal of Engineering Research and Technology, 8(5),* 112–116.

[2] Sharma, A., & Gupta, R. (2020). Automated leave and gate pass management for residential universities. *Journal of Computer Science and Information Technology, 6*(3), 45–51.

[3] Kumar, V., Reddy, P., & Nair, S. (2021). Mobile-based gate pass system with real-time approval tracking. *Proceedings of the International Conference on Computing and Communication Systems*, 214–219.

[4] Denso Wave Incorporated. (1994). *QR code: Development history and technical specification*. https://www.qrcode.com/en/history/

[5] Jayashankar, M., & Rao, K. V. (2020). Dynamic QR code generation for student attendance verification. *International Journal of Advanced Research in Computer Science, 11*(4), 88–93.

[6] Singh, H., Kaur, M., & Bhatia, T. (2022). QR-based entry management for institutional premises. *Journal of Information Security and Applications, 15*(2), 201–208.

[7] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *IEEE Computer, 29*(2), 38–47. https://doi.org/10.1109/2.485845

[8] Ferraiolo, D., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security, 4*(3), 224–274. https://doi.org/10.1145/501978.501980

[9] Ahmad, Z., & Hussain, M. (2021). Role-based access control implementation in web-based university management systems. *International Journal of Computer Applications, 183*(12), 34–40.

[10] Vercel Inc. (2024). *Next.js documentation — The React framework for the web*. https://nextjs.org/docs

[11] Meta Platforms, Inc. (2024). *React documentation*. https://react.dev

[12] Microsoft Corporation. (2024). *TypeScript documentation — JavaScript with syntax for types*. https://www.typescriptlang.org/docs

[13] Tailwind Labs Inc. (2024). *Tailwind CSS documentation — A utility-first CSS framework*. https://tailwindcss.com/docs

[14] NextAuth.js Contributors. (2024). *NextAuth.js documentation — Authentication for Next.js*. https://next-auth.js.org/getting-started/introduction

[15] MongoDB, Inc. (2024). *MongoDB documentation — The developer data platform*. https://www.mongodb.com/docs

[16] Mongoose. (2024). *Mongoose ODM documentation — Elegant MongoDB object modelling for Node.js*. https://mongoosejs.com/docs/guide.html

[17] Wirtz, D. (2023). *bcryptjs — Optimised bcrypt in JavaScript*. npm. https://www.npmjs.com/package/bcryptjs

[18] Recharts Group. (2024). *Recharts documentation — A composable charting library built on React*. https://recharts.org/en-US

[19] Minhazul Islam, M. (2023). *html5-qrcode — A cross-platform HTML5 QR code and bar code scanner*. GitHub. https://github.com/mebjas/html5-qrcode

[20] Vercel Inc. (2024). *Vercel documentation — Deploy and scale web applications*. https://vercel.com/docs

Appendix

This section contains supplementary technical material referenced throughout the report. It is intended to provide additional detail for readers who wish to examine the implementation more closely, without interrupting the flow of the main chapters.

Appendix A — Project Links

The GateVault application is available for live demonstration through the Vercel deployment link listed below. The complete source code, including all frontend components, backend API routes, and database models, is maintained in a public GitHub repository.

Resource	Link
Live Deployment	https://gatevault-agps.vercel.app
GitHub Repository	https://github.com/anandasaikiacse/gatevault
Dashboard (Direct)	https://gatevault-agps.vercel.app/dashboard

Appendix B — API Endpoint Reference

The following table lists all backend API endpoints implemented in GateVault. Each endpoint is accessible only to authenticated users whose session role matches the permitted roles specified. All endpoints communicate over HTTPS and return responses in JSON format.

Table B.1: GateVault API Endpoint Reference

Base URL: *https://gatevault-agps.vercel.app* · All endpoints communicate over HTTPS and return responses in JSON format.

Sl. No.	Endpoint	Method	Description	Permitted Role(s)
AUTHENTICATION				
1	/api/auth/[...nextauth]	GET POST	Handles all authentication operations — login, logout, and session validation — via NextAuth.js. The catch-all route pattern processes all NextAuth-specific sub-paths.	All Roles

REQUEST SUBMISSION AND RETRIEVAL				
2	/api/requests	POST	Creates a new Gate Pass or Leave request document in the database. The routing logic determines the approval path and sets the initial hodStatus and wardenStatus fields automatically on creation.	Student
3	/api/requests/student	GET	Retrieves all requests submitted by the currently authenticated student, scoped to their session. Returns active and historical records used to populate the student dashboard and history tab.	Student
4	/api/requests/hod	GET	Retrieves all Leave requests with hodStatus = pending, queued for the HoD's review. Gate Pass requests and already-actioned Leave requests are excluded from this response.	HoD
5	/api/requests/warden	GET	Retrieves all requests pending Warden action — Gate Pass requests with wardenStatus = pending, and Leave requests where hodStatus = approved and wardenStatus = pending.	Warden
6	/api/requests/all	GET	Retrieves the complete list of all request documents across all students and request types. Used exclusively by the Admin dashboard to provide a full system-wide view of activity.	Admin
APPROVAL ACTIONS AND VERIFICATION				
7	/api/requests/[id]	PATCH	Updates the approval status of a specific request identified by its database _id. Handles both HoD and Warden approval or denial actions. Triggers QR code generation when the Warden sets wardenStatus = approved.	HoDWarden
8	/api/requests/verify	GET	Looks up a request by its human-readable requestId (RID) and returns the current overallStatus along with student name and request details. Called in real time at the gate during QR scanning.	Security
USERS AND SCAN LOGS				
9	/api/users	GET	Retrieves the list of all registered users across all roles. Accessible exclusively by the Administrator to provide an overview of registered accounts. Password fields are excluded from the response payload.	Admin
10	/api/scans	POST	Records a new QR scan event in the Scans collection, capturing the requestId, scannedBy, statusAtScan, and scannedAt timestamp. Called automatically on each successful QR decode event at the gate.	Security

11	/api/scans/history	GET	Retrieves the complete scan history for the currently authenticated Security Guard. Returns all scan events recorded under their account, ordered by scannedAt descending, for display in the guard's session history log.	Security

HTTP Methods:

GET	Read / retrieve data
POST	Create a new resource
PATCH	Update an existing resource

Note — All endpoints validate the authenticated user's role server-side via NextAuth.js before executing any database operation. An unauthenticated or unauthorised request returns a 401 or 403 HTTP response respectively, without exposing any data.

Appendix C — Environment Configuration

The application uses environment variables to manage all sensitive configuration values. These variables are defined in a `.env.local` file during local development and are configured through the Vercel project settings dashboard in the production environment. No sensitive values are hardcoded in the source code or committed to the repository.

Table C.1: Required Environment Variables

Defined in `.env.local` for local development · Configured via Vercel Project Settings for production deployment.

Sl. No.	Variable Name	Required	Description	Format and Notes
1	MONGODB_URI Database connection	Yes	The full MongoDB Atlas connection string, including the cluster hostname, database name, and encoded credentials. This variable is read by the dbConnect() utility on every API route invocation to establish the database connection. Without this variable, the application cannot connect to MongoDB and will fail at runtime on any data operation.	**FORMAT** mongodb+srv://<user>:<password>@cluster.mongodb.net/<dbname>?retryWrites=true Obtained from the MongoDB Atlas dashboard under Database → Connect → Drivers. Credentials must be URL-encoded if they contain special characters.

2	NEXTAUTH_SECRET Session signing key	Yes	A cryptographically random secret string used by NextAuth.js to sign and verify JWT session tokens. This value must remain confidential at all times. If it is changed or missing, all existing user sessions are immediately invalidated and users are required to log in again. It is also used internally by NextAuth.js to encrypt the session cookie contents.	GENERATE VIA TERMINAL openssl rand -base64 32 Minimum 32 characters recommended. Must never be committed to version control or exposed in client-side code. Use a separate value for development and production environments.
3	NEXTAUTH_URL Application base URL	Yes	The canonical base URL of the deployed application. NextAuth.js uses this value to construct internal callback and redirect URLs for the authentication flow. It must match the actual domain from which the application is served exactly — including the protocol and, if applicable, the port number. An incorrect value will cause redirect errors during login and logout.	LOCAL DEVELOPMENT http://localhost:3000 PRODUCTION (VERCEL) https://gatevault-agps.vercel.app On Vercel, this variable is typically inferred automatically. Setting it explicitly is recommended to prevent misconfiguration on custom domain deployments.

Security Notice

All three variables contain sensitive credentials and must never be committed to the Git repository or exposed in any client-side code. The *.env.local* file is excluded from version control via the project's *.gitignore* configuration. For production, all variables are stored securely in the Vercel project's encrypted environment settings and injected at build time.

Appendix D — Sample Database Documents

The following JSON excerpts illustrate the structure of documents stored in each MongoDB collection, as they appear in the database after a complete request lifecycle.

Sample User Document — Student

json


```
{
  "_id": "64f3a1c2e4b0c12d3f8a9b01",
  "name": "Ananda Saikia",
  "email": "et22bthcs072@kazirangauniversity.in",
  "password": "$2b$10$hashed_password_string",
  "role": "student",
  "studentId": "ET22BTHCS072",
  "department": "Computer Science and Engineering",
  "hostelRoom": "B-313",
  "createdAt": "2026-02-01T09:00:00.000Z"
}
```


Sample Request Document — Approved Gate Pass

json


```
{
  "_id": "64f3b2d3e4b0c12d3f8a9c02",
  "requestId": "RID-20260316-0042",
  "studentId": "64f3a1c2e4b0c12d3f8a9b01",
  "studentName": "Ananda Saikia",
  "type": "gate_pass",
  "reason": "GATE exam",
  "destination": "Jorhat",
  "fromDate": "2026-03-16T10:00:00.000Z",
  "toDate": "2026-03-16T18:00:00.000Z",
  "hodStatus": "not_required",
  "wardenStatus": "approved",
  "wardenApprovedBy": "64f3a1c2e4b0c12d3f8a9b07",
  "overallStatus": "approved",
  "qrCodeData": "RID-20260316-0042",
  "createdAt": "2026-03-16T08:30:00.000Z",
  "updatedAt": "2026-03-16T09:15:00.000Z"
}
```

Sample Scan Document

json


```
{
  "_id": "64f3c4e5e4b0c12d3f8a9d03",
  "requestId": "64f3b2d3e4b0c12d3f8a9c02",
  "scannedBy": "64f3a1c2e4b0c12d3f8a9b09",
  "statusAtScan": "approved",
  "scannedAt": "2026-03-16T10:05:00.000Z"
}
```


Appendix E — List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
RBAC	Role-Based Access Control
QR	Quick Response
RID	Request ID
JWT	JSON Web Token
ODM	Object Data Modelling
DFD	Data Flow Diagram
ER	Entity Relationship
HoD	Head of Department
UI	User Interface
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
npm	Node Package Manager
CSS	Cascading Style Sheets
NIST	National Institute of Standards and Technology
ERP	Enterprise Resource Planning